Game Engine Concepts

Fachhochschule Oberösterreich

2

Game Engine Basics


Spacehuddle

Basics

What is a game?

- We all have a pretty good intuitive notion of what a game is
- **“Game” encompasses**
  - Board games like chess and monopoly
  - Card games like poker and blackjack
  - Casino games like roulette and slot machines
  - Military war games
  - Computer games
  - Play among children
- **In academia: game theory**
  - Multiple agents select strategies and tactics in order to maximize their gains within the framework of well-defined set of game rules
- Computer-based entertainment: three-dimensional virtual world with a main character under player control

Basics

What is a game?

- From a **computer science** perspective:
  - **Games** are **soft real-time interactive agent-based computer simulations**
- Let's break this down:
  - The concept of **time** is an important aspect in games
    - Real-time systems, at their core, implement **deadlines**
  - For **"soft"** real-time systems, missed deadlines are not catastrophic
  - Subset of the real or an imaginary **world is modeled mathematically** -> can be manipulated by a computer
  - A model is a **simplification of reality** (or imaginary reality) -> simulation
  - **Agents:** distinct entities interact with each other
    - Examples: vehicles, characters, fireballs, power-ups etc.
  - Most computer games are implemented in an **object-oriented programming language**
    - (we will learn more about OOP in the next semesters)
  - Interactive video games are **temporal simulations** -> virtual game world is dynamic -> state of the game world changes over time
  - **Interactivity:** **Input** from human player(s)

Basics

What is a game?

- The concept of **time** is an important aspect in games
- **Examples:**
  - Video games need the screen to be updated at least 24 times per second (illusion of motion)
  - Most games render at 30—60 frames per second (based on NTSC monitor's refresh rate)
  - Physics simulations need to be updated 120 times per second in order to remain stable
  - A character's AI system may need to "think" at least once every second to prevent "stupid" behavior
  - Audio buffers need to be filled once every 1/60 second to prevent audible glitches

- **Hard Real-Time**
  - Small data files/packages
  - Response times in milliseconds
  - Performance is predictable
  - Safety is critical
  - Examples: Flight Control, Railway Signaling System, Satellite Launch, Driving Physics Simulation, etc.

- **Soft Real-Time**
  - Larger files and data packages
  - Higher response times
  - Not safety critical – missing
  - Examples: Media Electronics, Game Engines, etc.

Basics

What is a game?

- Simulations are implemented by **running calculations repeatedly** to determine the state of the system at each discrete time step
- Main **game loop** updates:
  - Game logic
  - Physics simulations
- **Results** are rendered by
  - Displaying graphics
  - Emitting sound
  - Producing force-feedback on the joy-con

Basics

What is a game engine?

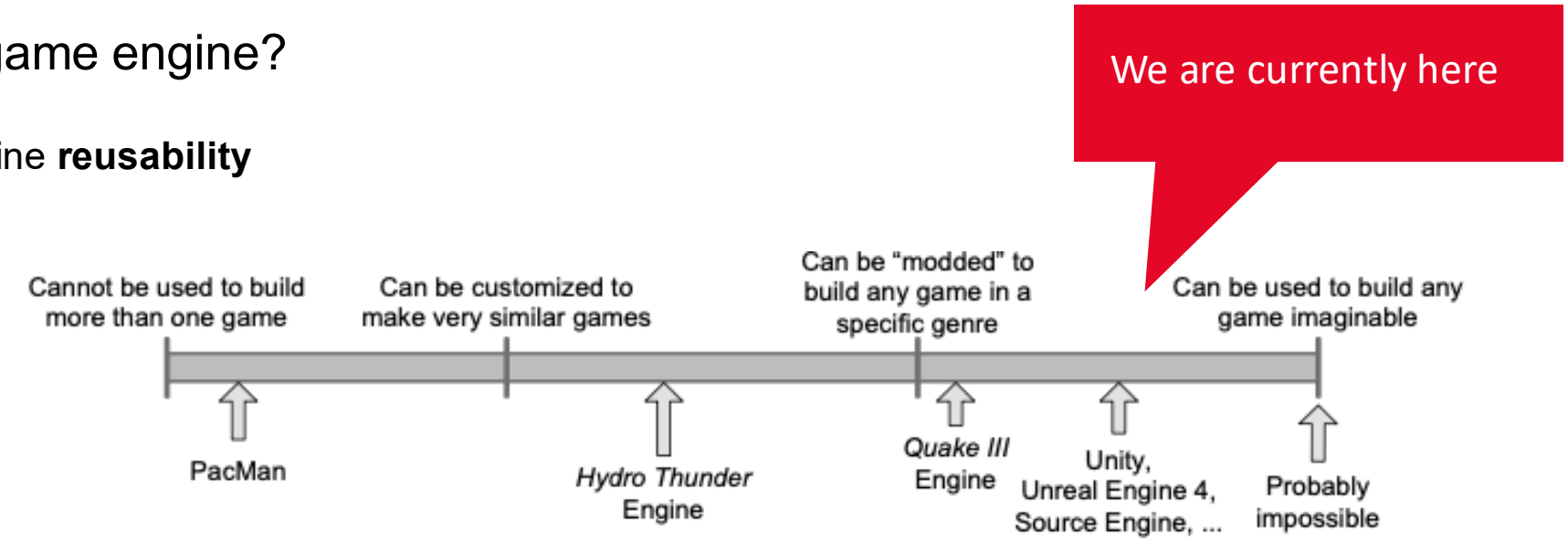
- Term “game engine” first used in mid-1990s
  - First-person shooter (FPS) like Doom by id Software
  - Separation between its core software components (e.g., 3D graphics rendering, collision detection or audio system) and the art assets, game worlds, and rules of play
- Birth of “mod community”
  - Building new games by modifying existing games

- Space Huddle: What games mods do you know (or even mod yourself)?

Basics

What is a game engine?

- Game engine **reusability**



- Game engines are usually carefully crafted to run a **particular game** on a **particular hardware** platform
- General-purpose multiplatform engines usually run a **particular genre**
  - FPS Maker, RPG Maker
- The more general a game engine is, the less optimal it is for running a particular game on a particular platform
- Trade-offs between reusability and specialization

Basics

Game Genres

- Game engine differences across genres

- Let's look at three different game genres
  - Massively multiplayer online game (MMOG)
  - First-person shooter (FPS)
  - Real-time strategy (RTS)

- What do these games have in common?
  - 3D graphics
  - User input

Basics

Game Genres

- **First-Person Shooter (FPS)**

- Examples
  - Quake
  - Unreal Tournament

- Core technologies
  - Illusion of immersion in a hyperrealistic world
  - efficient rendering of large 3D virtual worlds
  - a responsive camera control/aiming mechanic
  - high-fidelity animations of the player’s virtual arms and weapons
  - a wide range of powerful handheld weaponry
  - high-fidelity animations and artificial intelligence

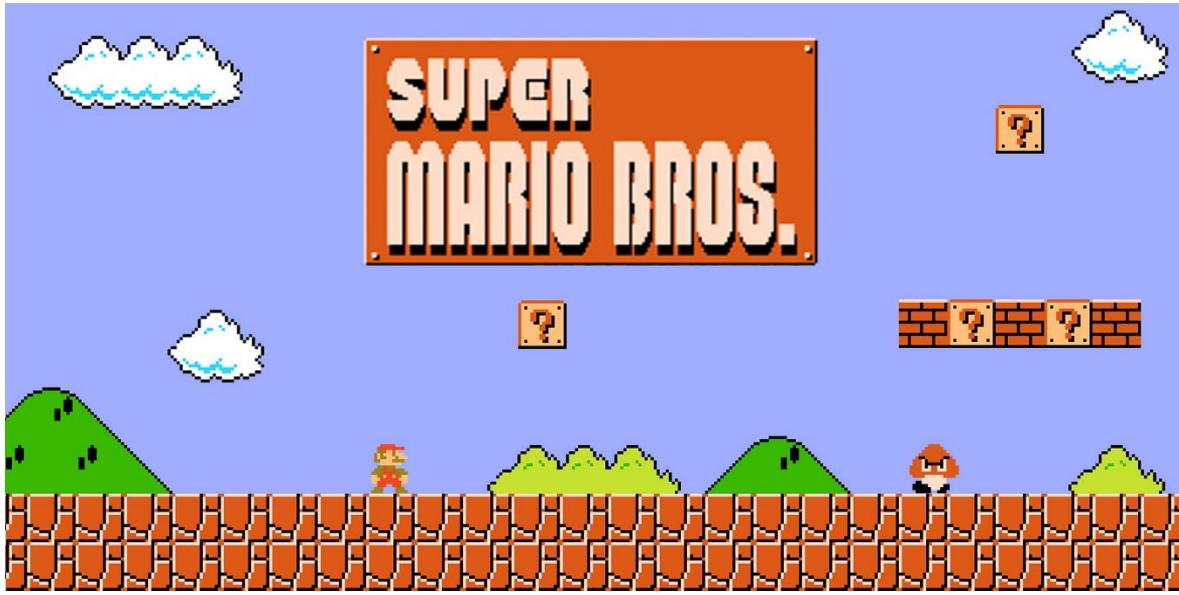
Basics

Game Genres

- **Platformers and Other Third-Person Games**

- Examples:
  - Super Mario Bros
  - Donkey Kong

- Core technologies
  - moving platforms, ladders, ropes, trellises and other interesting locomotion modes
  - puzzle-like environmental elements
  - a third-person “follow camera” which stays focused on the player character
  - a complex camera collision system for ensuring that the view point never “clips” through background geometry



Basics

Game Genres

- **Fighting Games**

- Examples
  - Tekken
  - Soul Calibur

- Core technologies
  - a rich set of fighting animations
  - accurate hit detection
  - a user input system capable of detecting complex button and joystick combinations
  - crowds, but otherwise relatively static backgrounds
  - realistic skin shaders with subsurface scattering and sweat effects
  - photo-realistic lighting and particle effects
  - physics-based cloth and hair simulations for the characters.

Basics

Game Genres

- **Racing Games**

- Examples
  - Simulation-focused: Gran Turismo
  - Arcade racers: San Francisco Rush

- Core technologies
  - “Tricks” when rendering distant background elements, such as 2D cards for trees, hills and mountains
  - Tracks are broken down into simple 2D regions (“sectors”)
  - Camera follows behind the vehicle for a third-person perspective or inside for a first-person perspective
  - Ensuring that the camera does not collide with background geometry, tunnels or other tight spaces (e.g., garages)
  - Motion blur effects for speed illusion

Basics

Game Genres

- **Strategy Games**

- Examples
  - Age of Empires
  - Starcraft

- Core technologies
  - Each unit is relatively low-res, so that the game can support large numbers of them on-screen at once
  - Height-field terrain is usually the canvas upon which the game is designed and played.
  - The player is often allowed to build new structures on the terrain in addition to deploying his or her forces.
  - User interaction is typically via single-click and area-based selection of units
  - Menus or toolbars containing commands, equipment, unit types, building types, etc.

Basics

Game Genres

- **Massively Multiplayer Online Games (MMOG)**

- Examples
  - World of Warcraft
  - Fortnite

- Core technologies
  - Powerful battery of servers
  - Servers maintain the authoritative state of the game world, manage user signing in and out of the game, inter-user chat, etc.
  - Often subscription-based revenues and micro-transactions
  - Lower graphics fidelity

Basics

Game Genres

- **Player-Authored Content**

- Examples
  - Minecraft
  - LittleBigPlanet

- Core technologies
  - Simplicity!
  - Create, modify, share own content with the game world
  - In-game editors to build and shape worlds, levels, objects
  - Marketplace allows players to share and monetize their creations
  - Collaborations in real-time

Basics

Game Genres

- **Virtual, Augmented, and Mixed Reality**

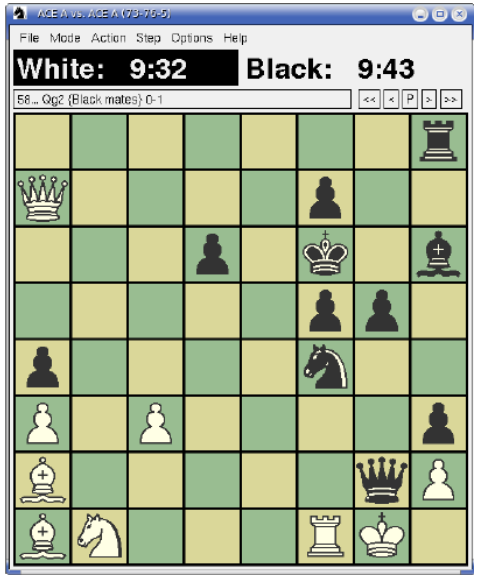
- Examples
  - Pokemon Go
  - Job Simulator

- Core technologies
  - Stereoscopic rendering: rendering the scene twice (once for each eye)
    - For head-mounted devices (HMDs) like the Meta Quest 3
  - Very high frame rate necessary, otherwise users will experience disorientation, nausea, etc.
  - Navigation issues: not being able to physically walk (VR mode)
  - Smartphone-based AR games often make use of the user's location

Basics

Game Genres

- **Other Genres**
  - Role-Playing Games (RPG)
  - Sports
  - God games
  - Environmental/social simulation games
  - Puzzle games
  - Conversions of non-electronic games
  - Web-based games
  - …



Basics

Game Engines

- **Quake Family of Engines**
  - World's first 3D FPS: Castle Wolfenstein 3D (1992)
  - Written by id Software for the PC platform
  - Further titles: Doom, Quake, Quake II, Quake III
  - All of these **engines are very similar in architecture**

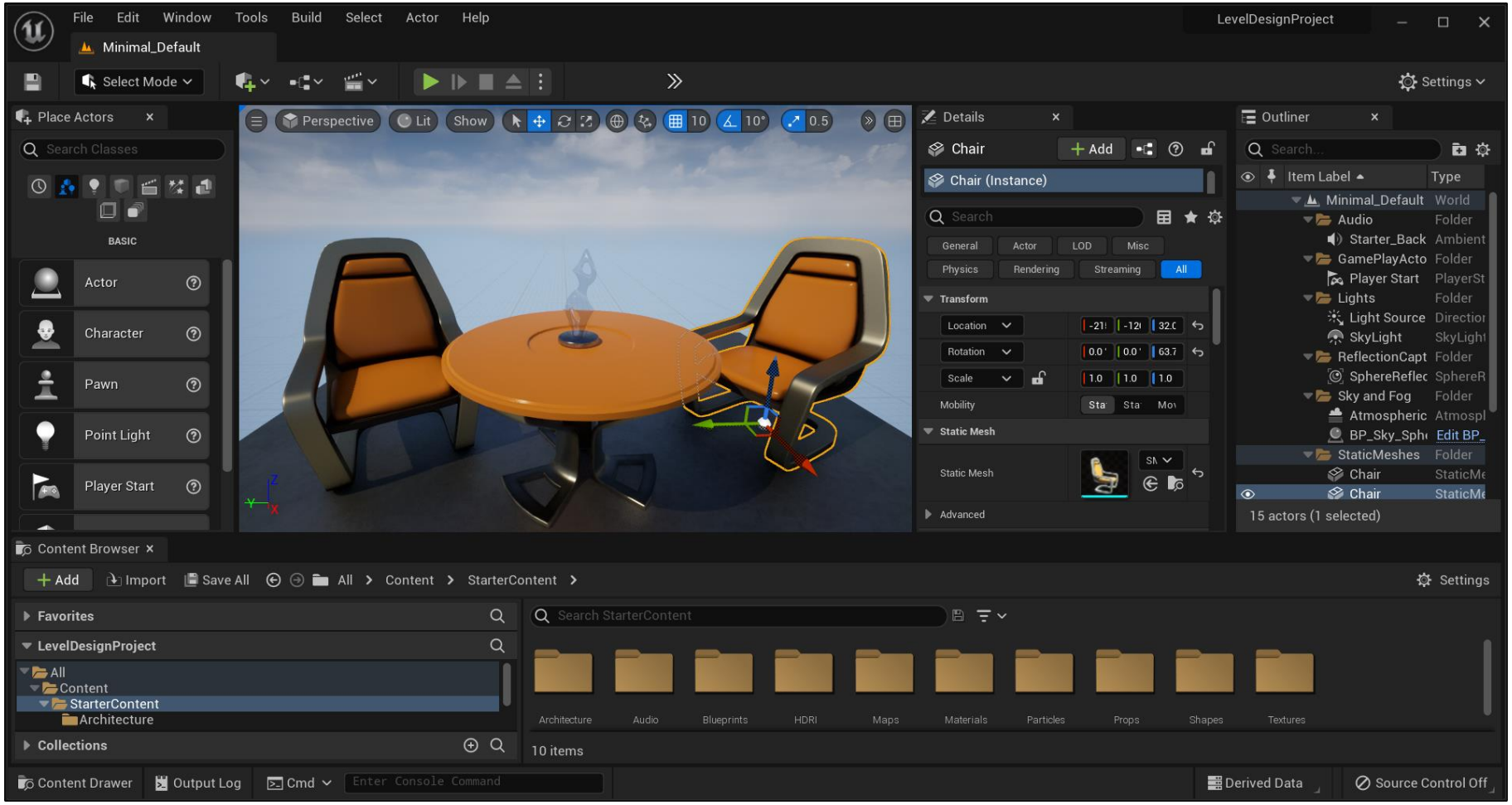
Basics

Game Engines

We will dive deep into Unreal Engine in the next semesters, stay tuned…

→ **Unreal Engine**

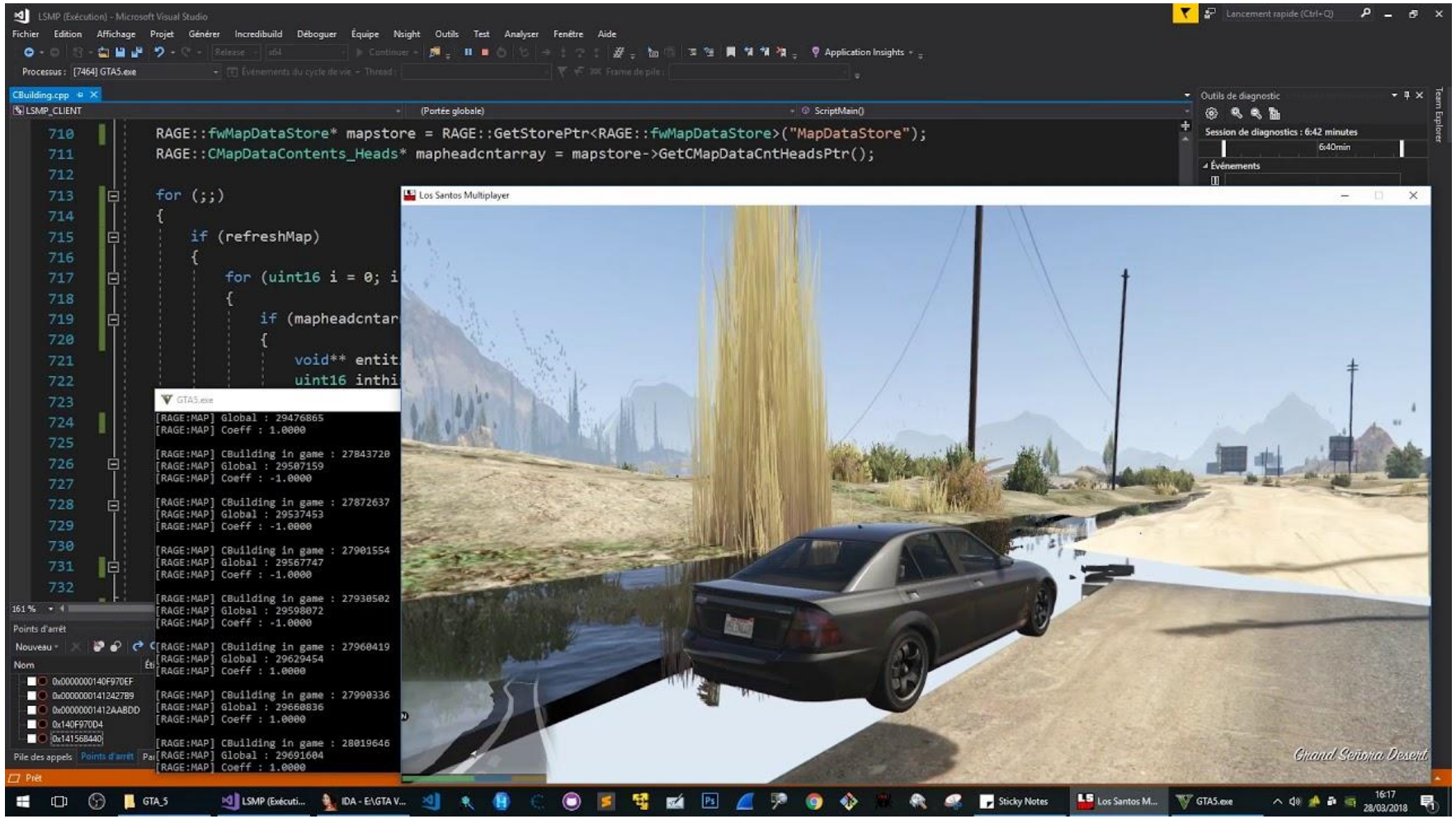
UNREAL ENGINE



Basics

Game Engines

→ **Rockstar Advanced Game Engine (RAGE)**

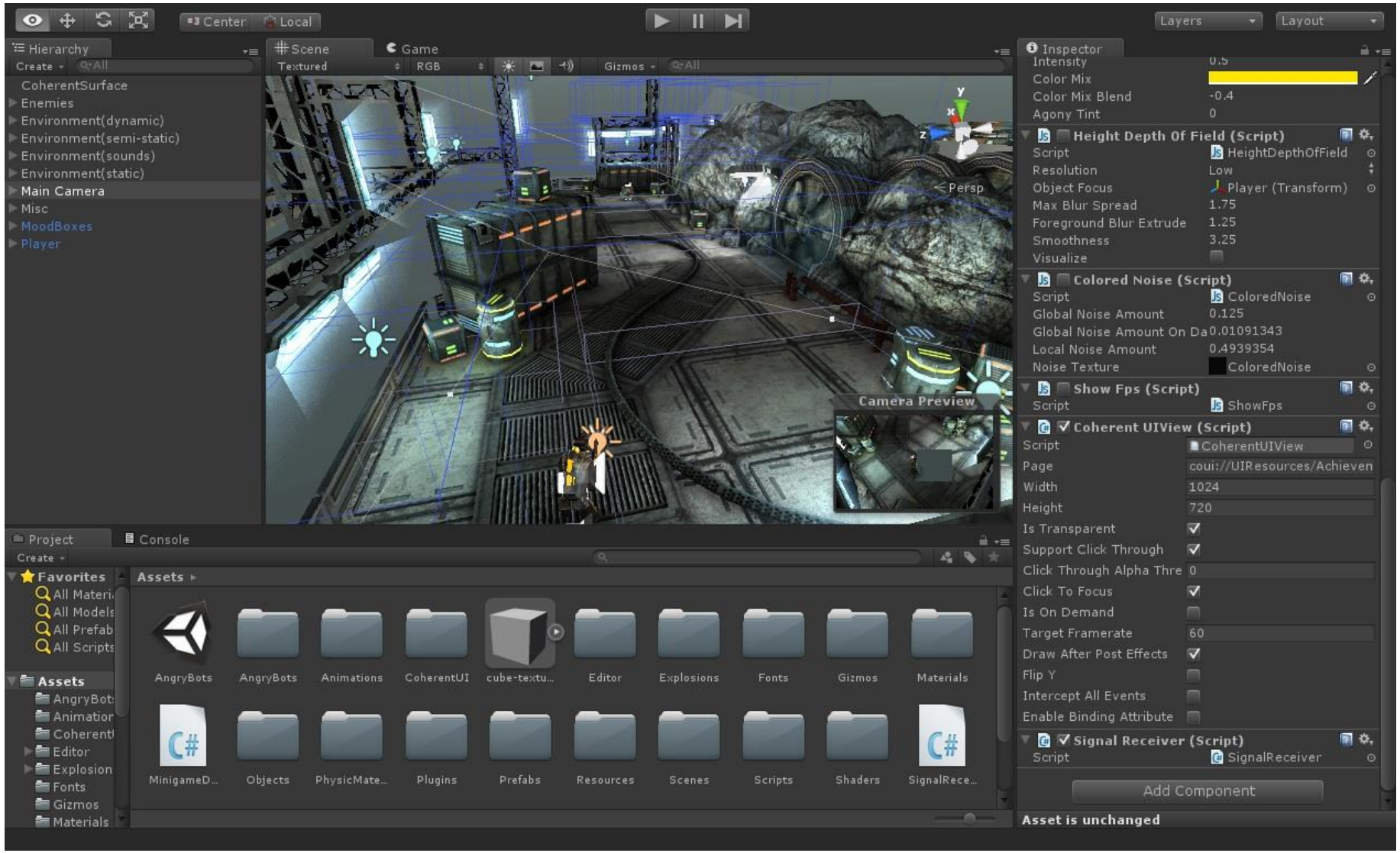


Basics

Game Engines

We will dive deep into Unity in this course!

- **Unity**

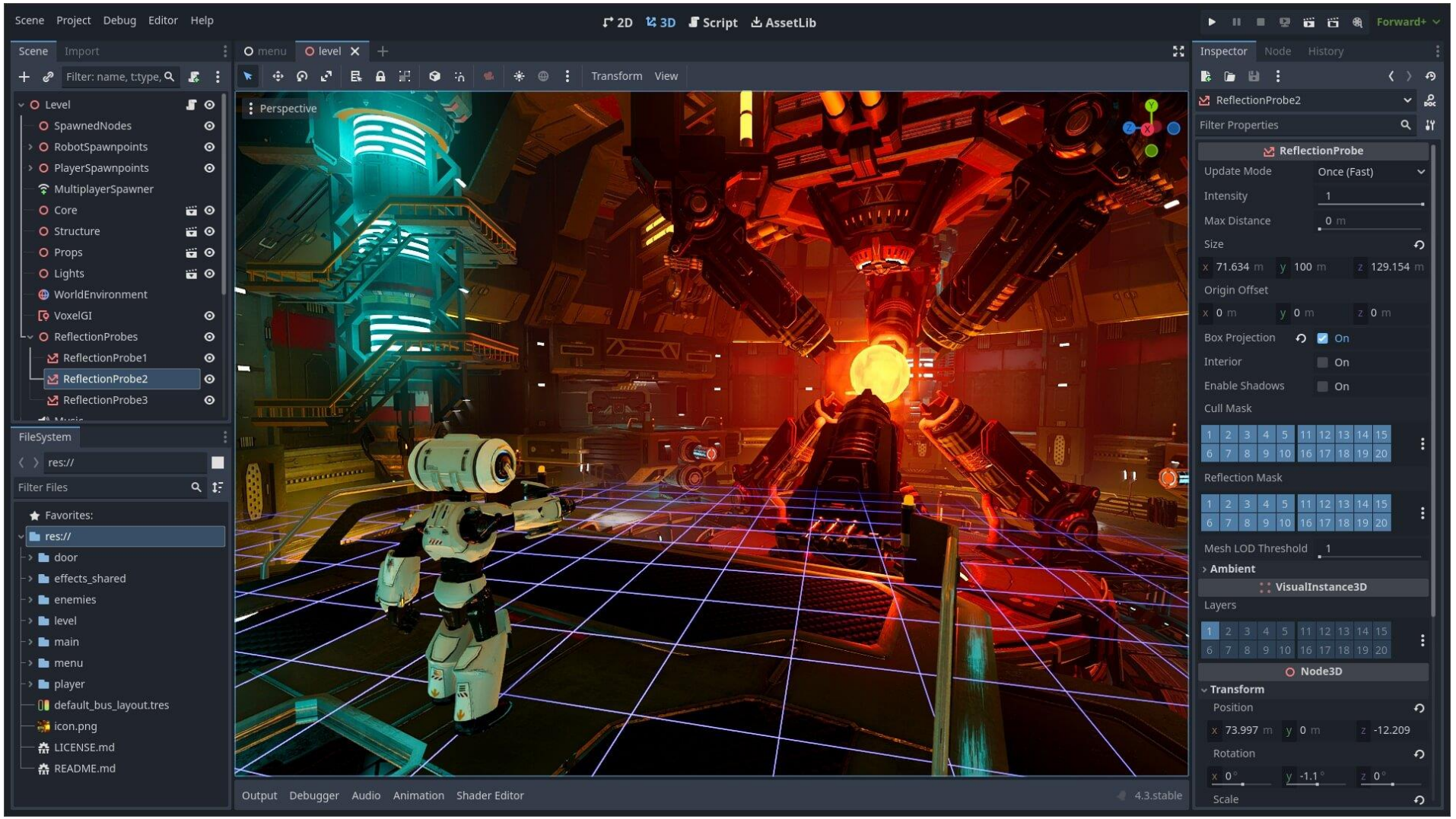


Basics

Game Engines

We will dive deep into Godot in this course!

- **Godot**



Basics

Game Engines

- **Other Game Engines**
  - Felgo (formerly V-Play)
    - Developed by former FH Hagenberg students
    - Previously focus on games, now on mobile apps
  - LeadWerks engine
  - Irrlicht engine
  - Panda3D
  - Torque
  - …

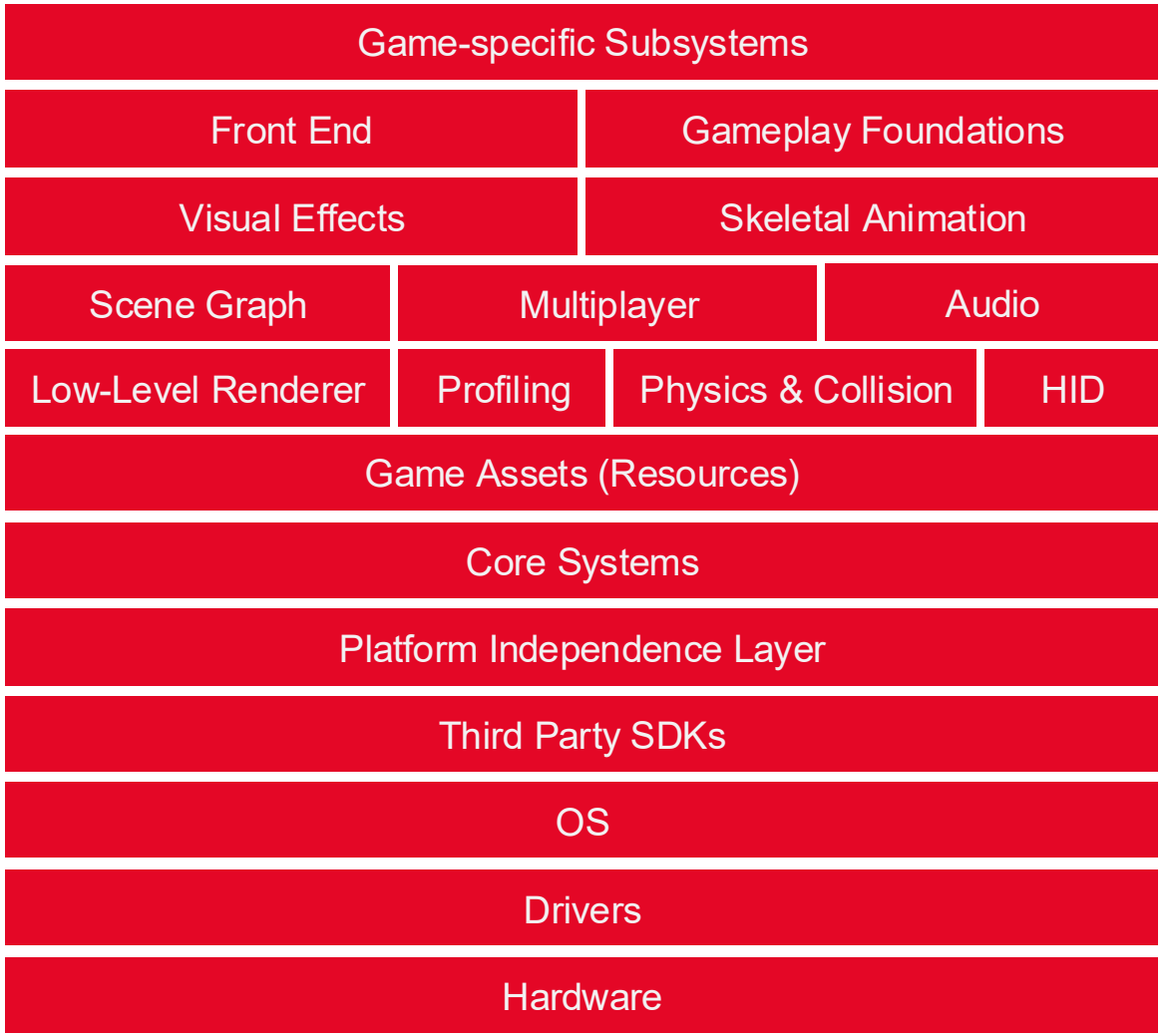
Basics

Runtime Engine Architecture

→ **Overview**

→ Game engines are **large software systems**

- **2 components**
  - **Runtime** = “engine”: executes the game
  - **Tool suite:** editor, asset pipelines, dev tools
- Built in **layers**
- Upper layers depend on lower layers

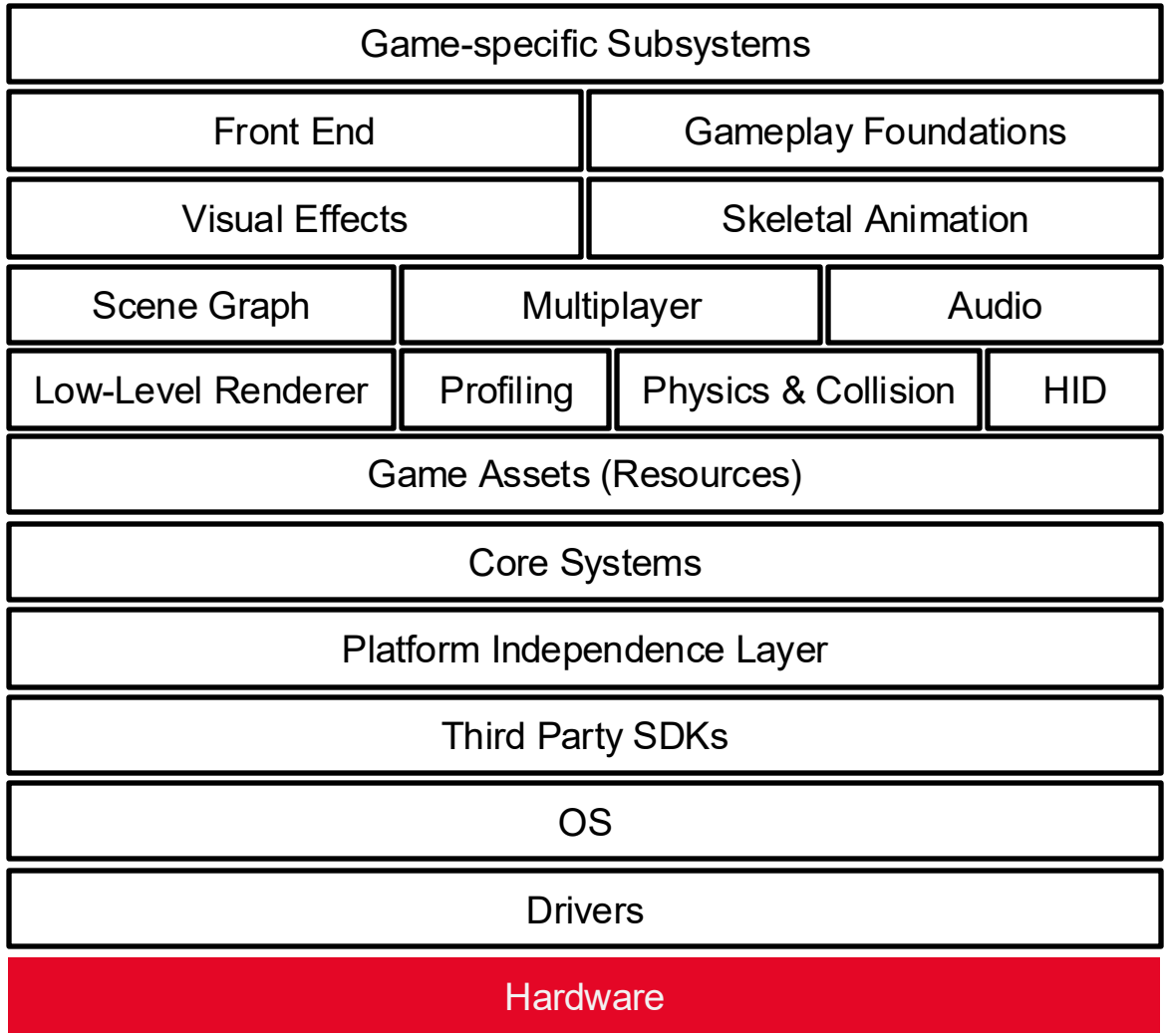


Basics

Runtime Engine Architecture

→ **Target Hardware**

- Represents the **physical system** on which the game runs
  - PCs (Windows, Linux, macOS)
  - Mobile devices (iOS, Android)
  - Consoles (PlayStation, Xbox, Nintendo Switch, etc.).
- **Platform Diversity:** Each platform has unique hardware constraints, operating systems, and APIs that influence engine design and optimization
- **Platform-Agnostic:** Most engine concepts apply across all platforms, but certain architectural decisions must adapt to specific PC, console, or mobile requirements

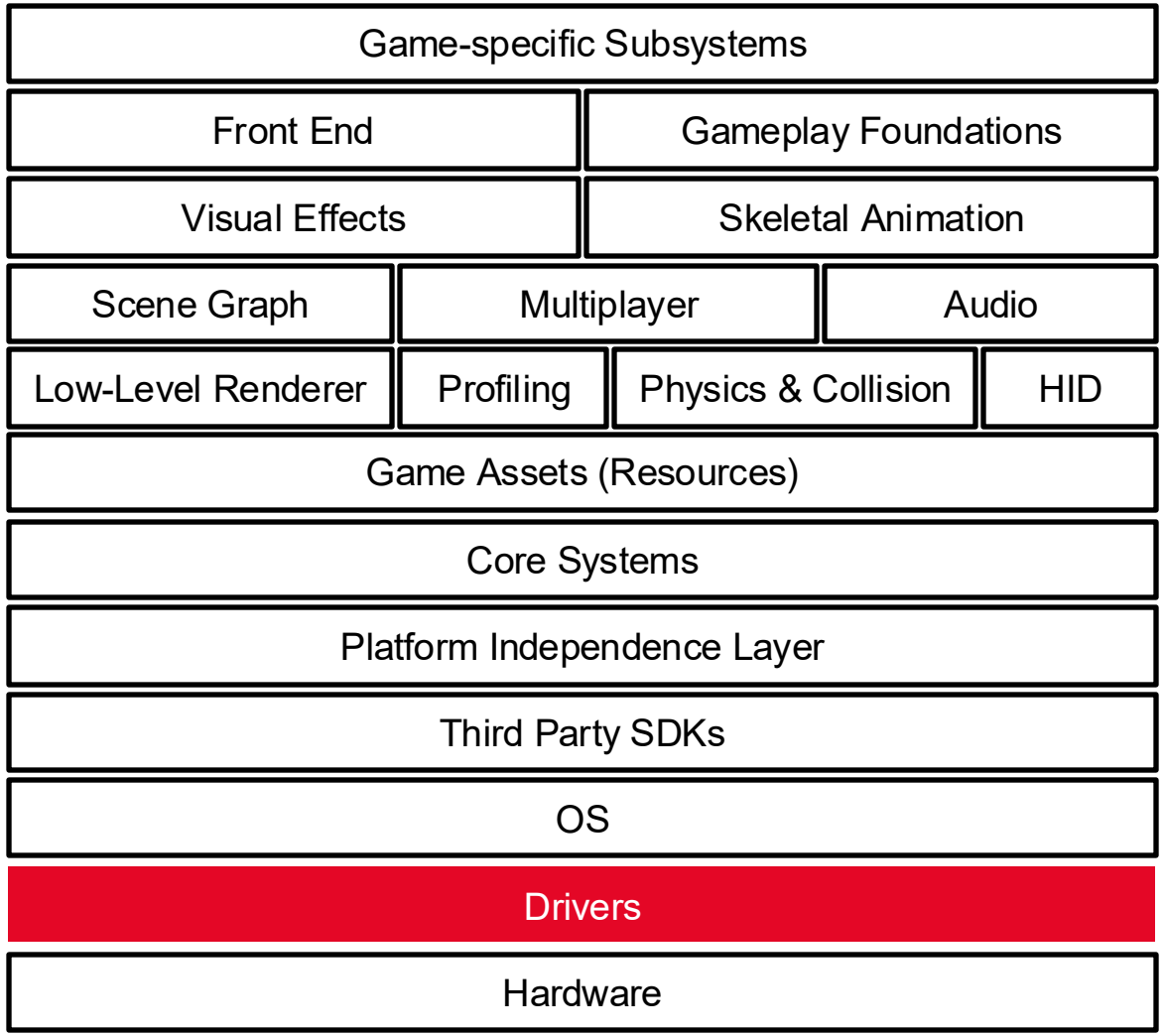


Basics

Runtime Engine Architecture

→ **Device Drivers**

- Low-level software that **manages and controls hardware components** (e.g., GPU, audio, input devices).
- **Abstraction:** They hide hardware-specific details, allowing the operating system and game engine to interact with diverse hardware in a consistent, unified way.

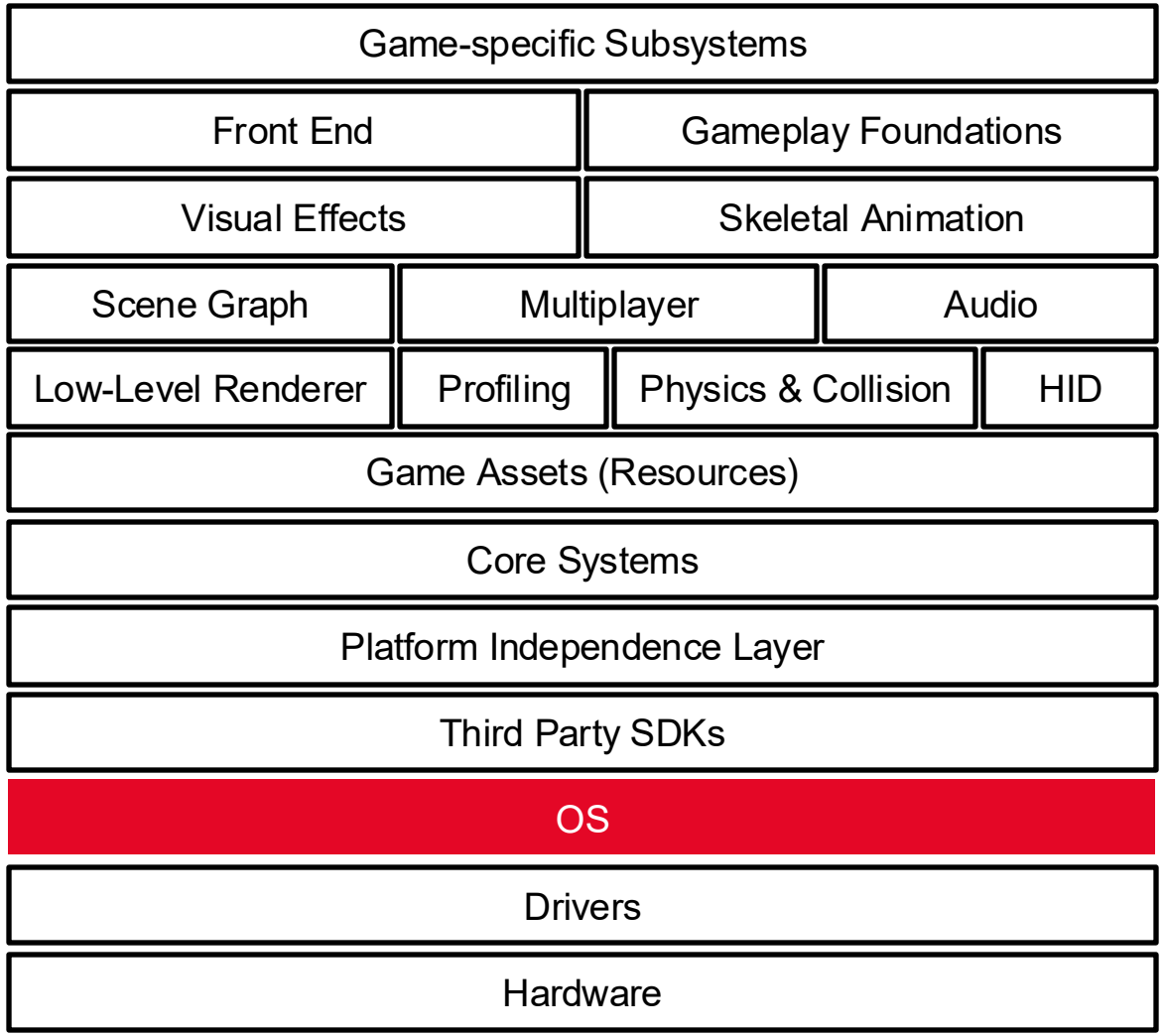


Basics

Runtime Engine Architecture

→ Operating System (OS)

- **System Coordination:** The OS manages all running programs and allocates hardware resources through preemptive multitasking, meaning the game must share the system and cannot assume full hardware control.
- **PC vs. Console Evolution:** Early consoles gave games total hardware ownership, but modern consoles (PS5, Xbox One, etc.) now run full OS environments with background services and multitasking—similar to PCs.
- **Convergence Trend:** The architectural gap between PC and console systems is shrinking as both now rely on complex, always-running operating systems.

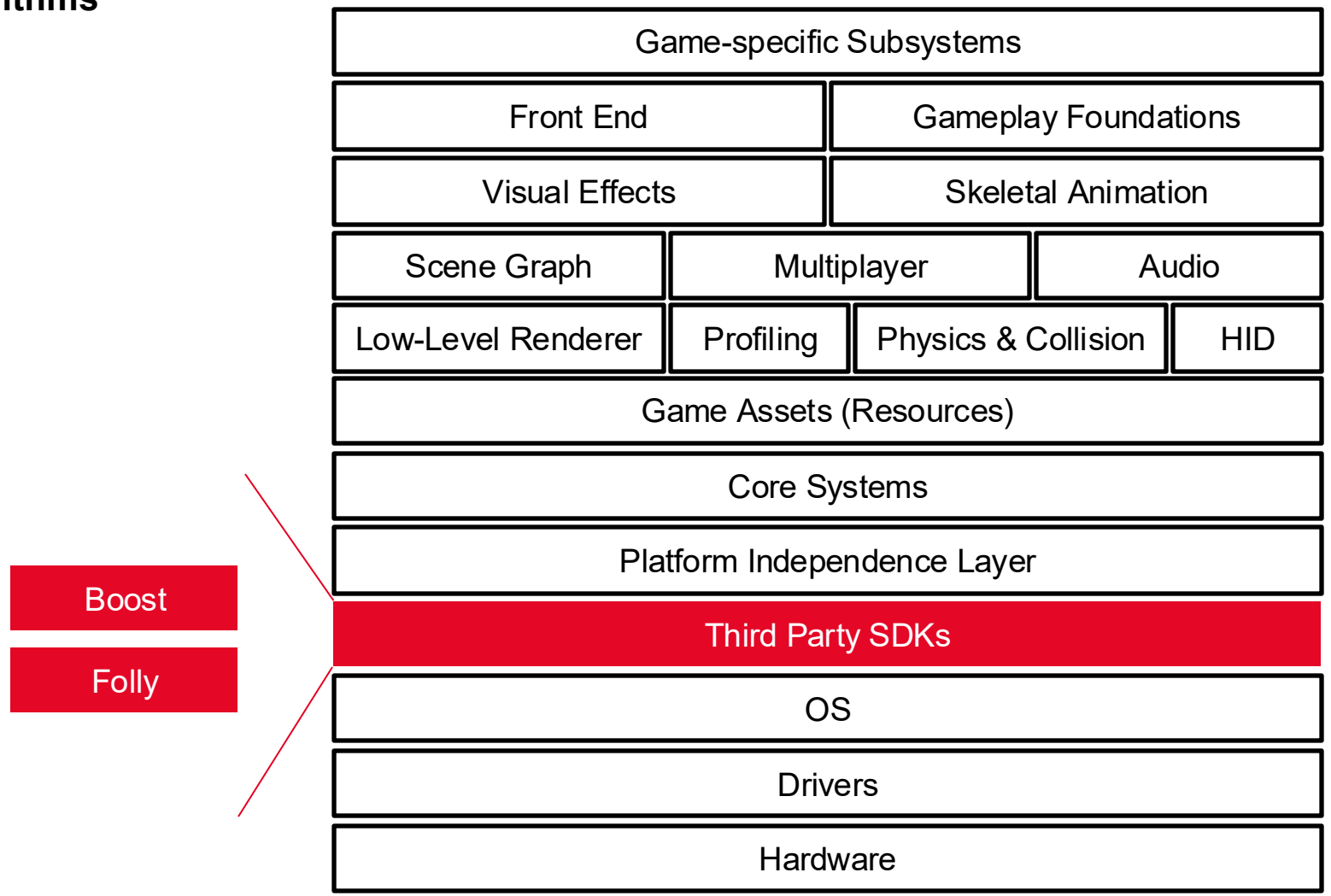


Basics

Runtime Engine Architecture

→ **Third-Party SDKs and Middleware: Algorithms**

- **Data Structures**
  - Vectors
  - Lists
  - Trees
  - Graphs
- **Algorithms**
  - Pathfinding
  - Logging

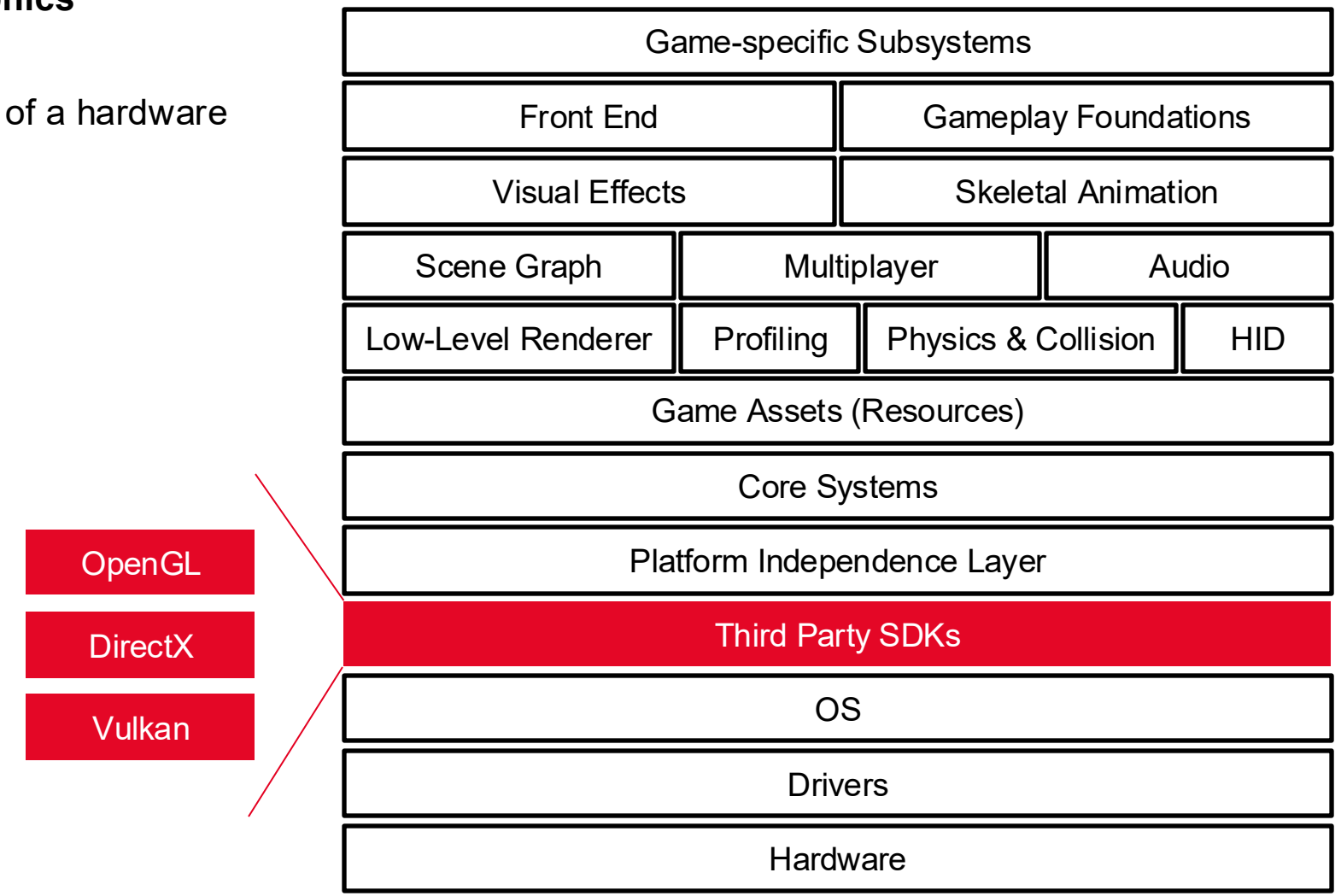


Basics

Runtime Engine Architecture

→ **Third-Party SDKs and Middleware: Graphics**

- Game rendering engines are built on top of a hardware interface library

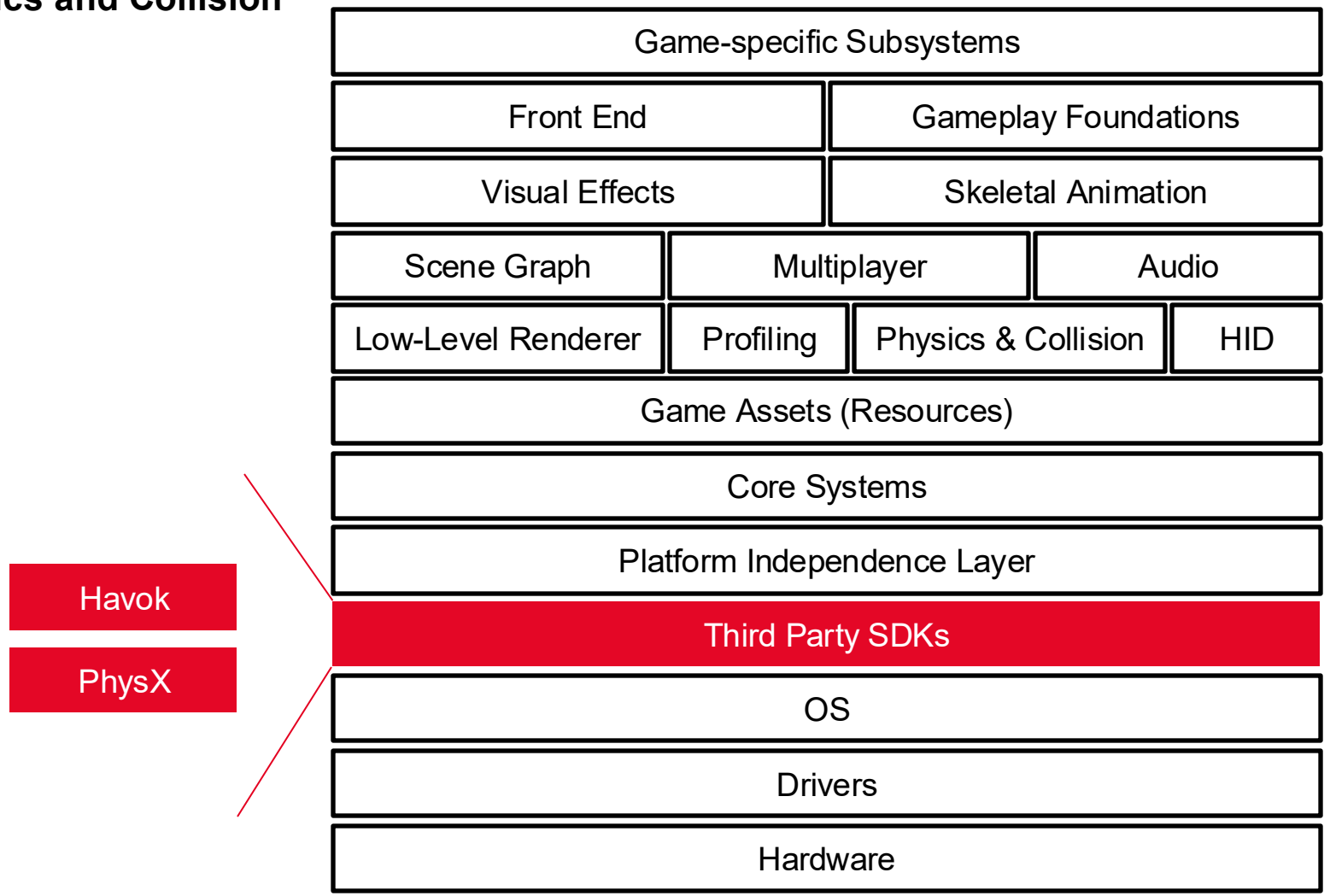


Basics

Runtime Engine Architecture

→ **Third-Party SDKs and Middleware: Physics and Collision**

- Collision detection
- Rigid body dynamics (“physics”)

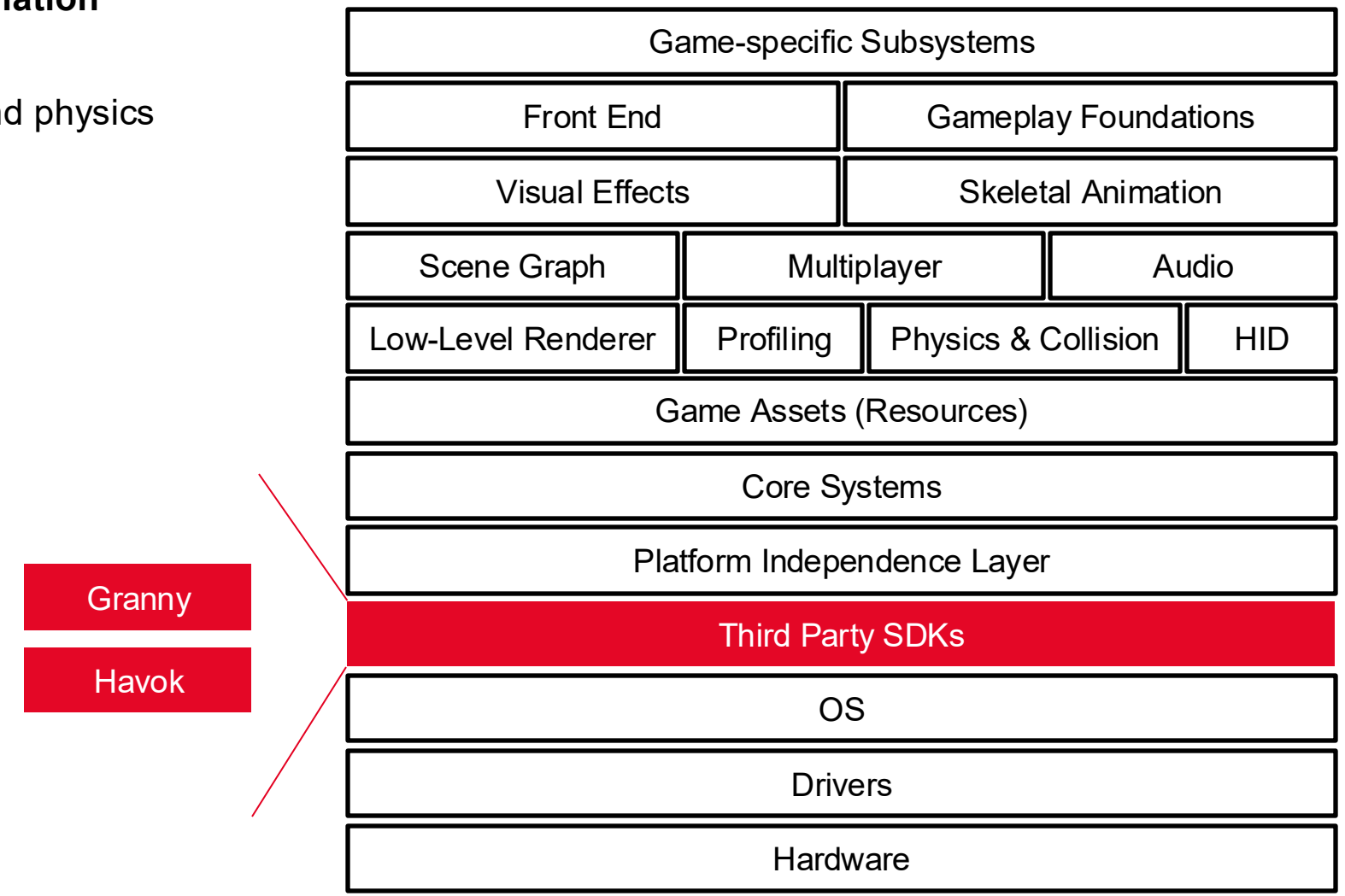


Basics

Runtime Engine Architecture

→ **Third-Party SDKs and Middleware: Animation**

- The line between character animation and physics is beginning to blur

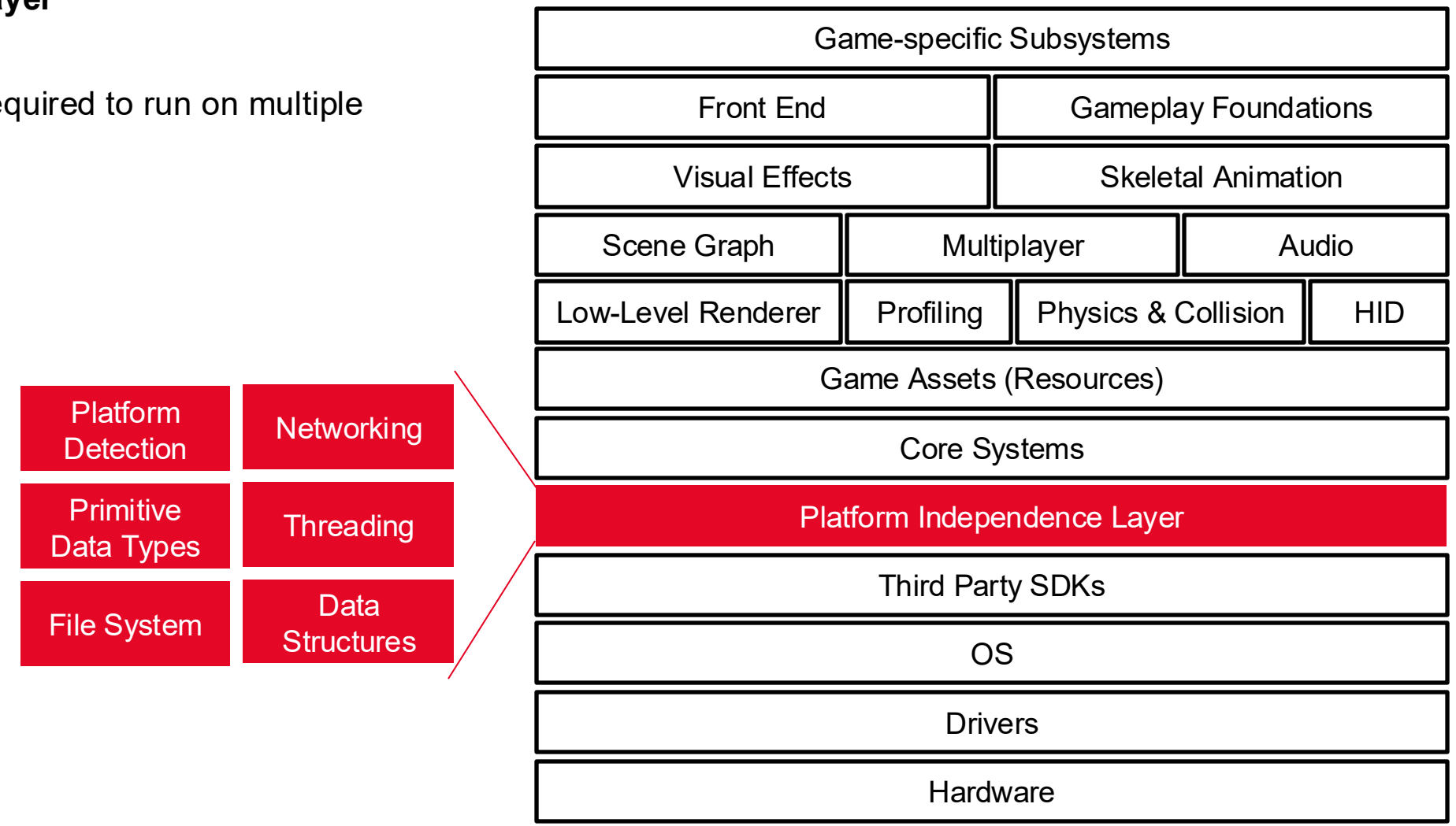


Basics

Runtime Engine Architecture

→ **Platform Independence Layer**

- Most game engines are required to run on multiple hardware platforms

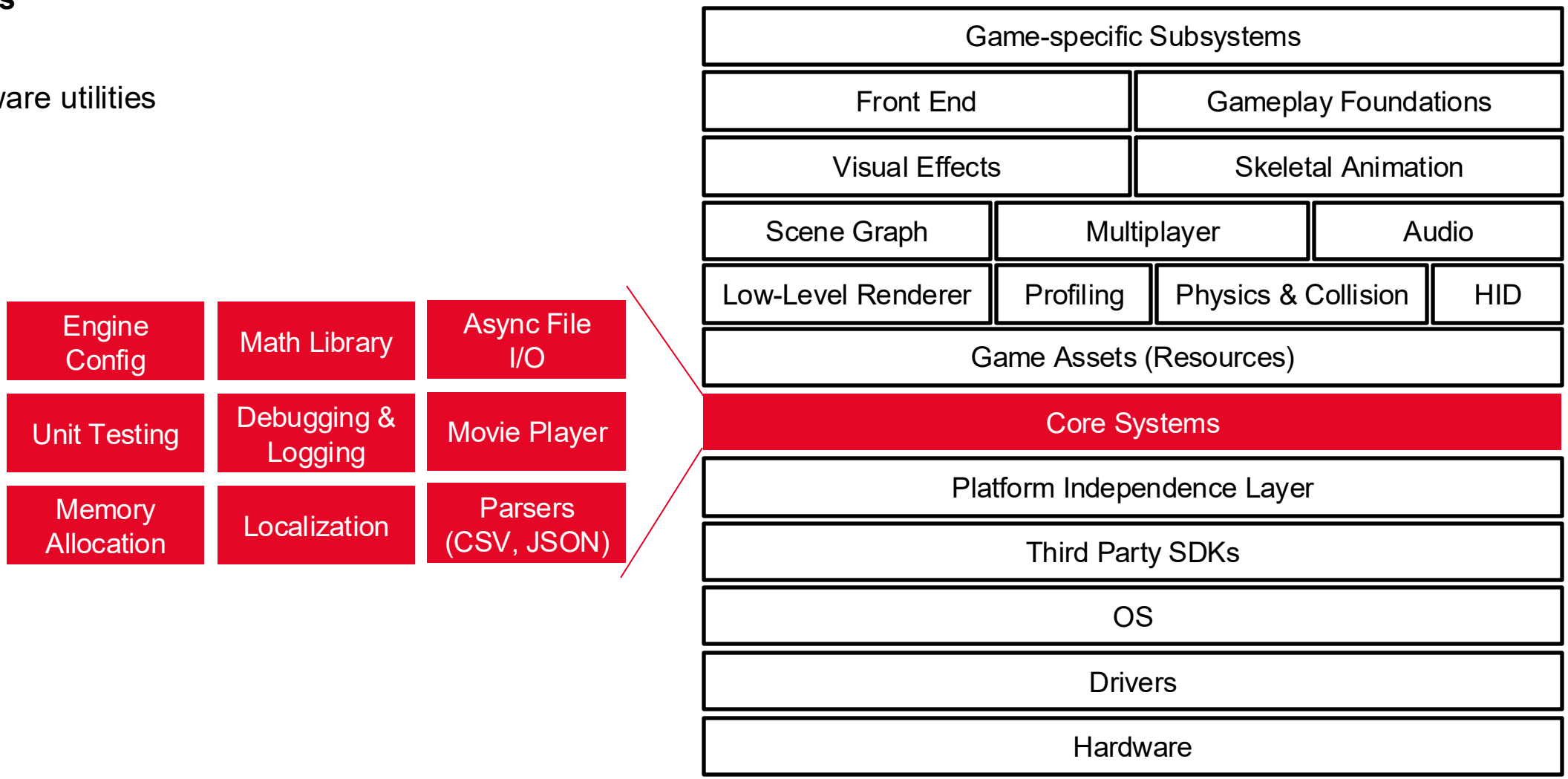


Basics

Runtime Engine Architecture

→ **Core Systems**

- Useful software utilities

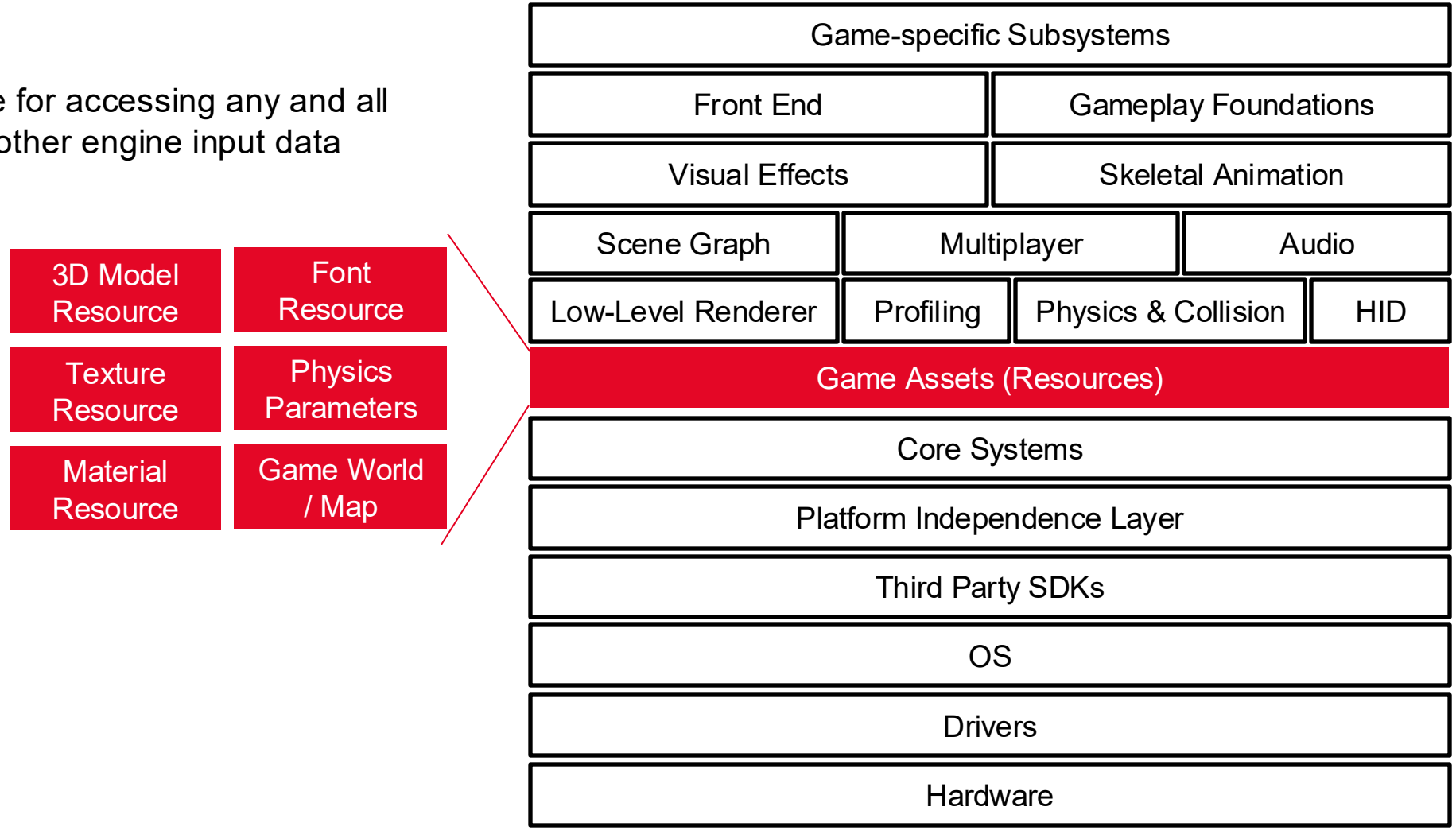


Basics

Runtime Engine Architecture

→ Resource Manager

- Provides a unified interface for accessing any and all types of game assets and other engine input data

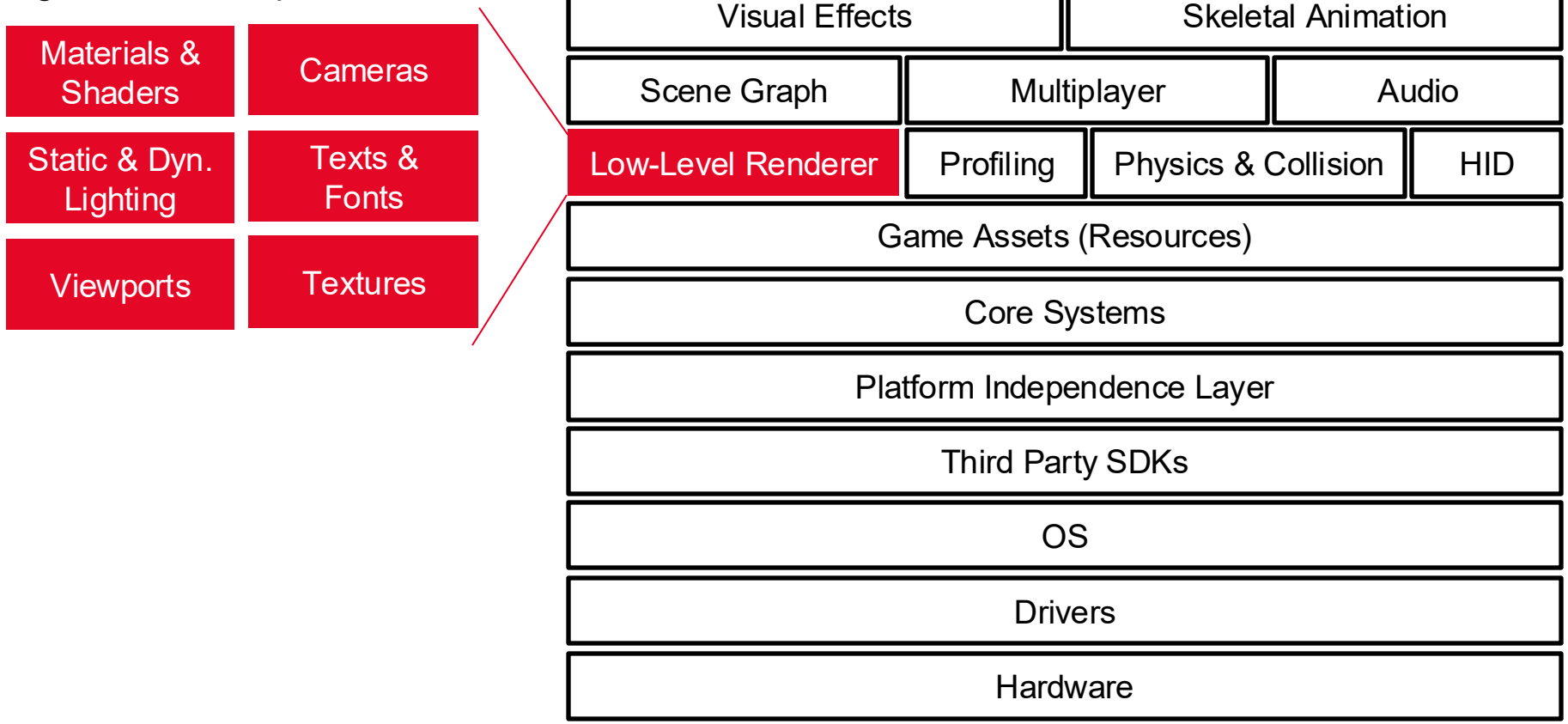


Basics

Runtime Engine Architecture

→ Rendering Engine

- Rendering a collection of **geometric primitives** as quickly as possible without regard for which portions of a scene may be visible

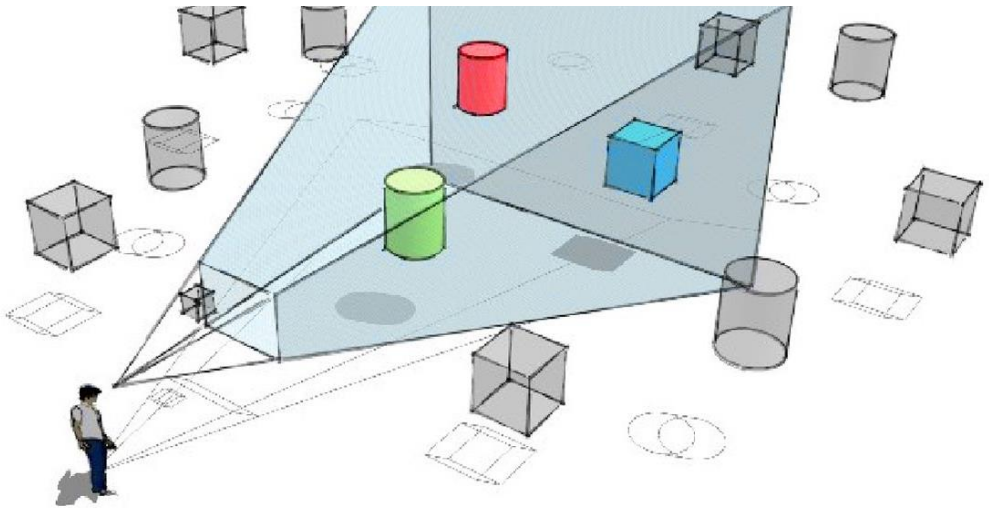


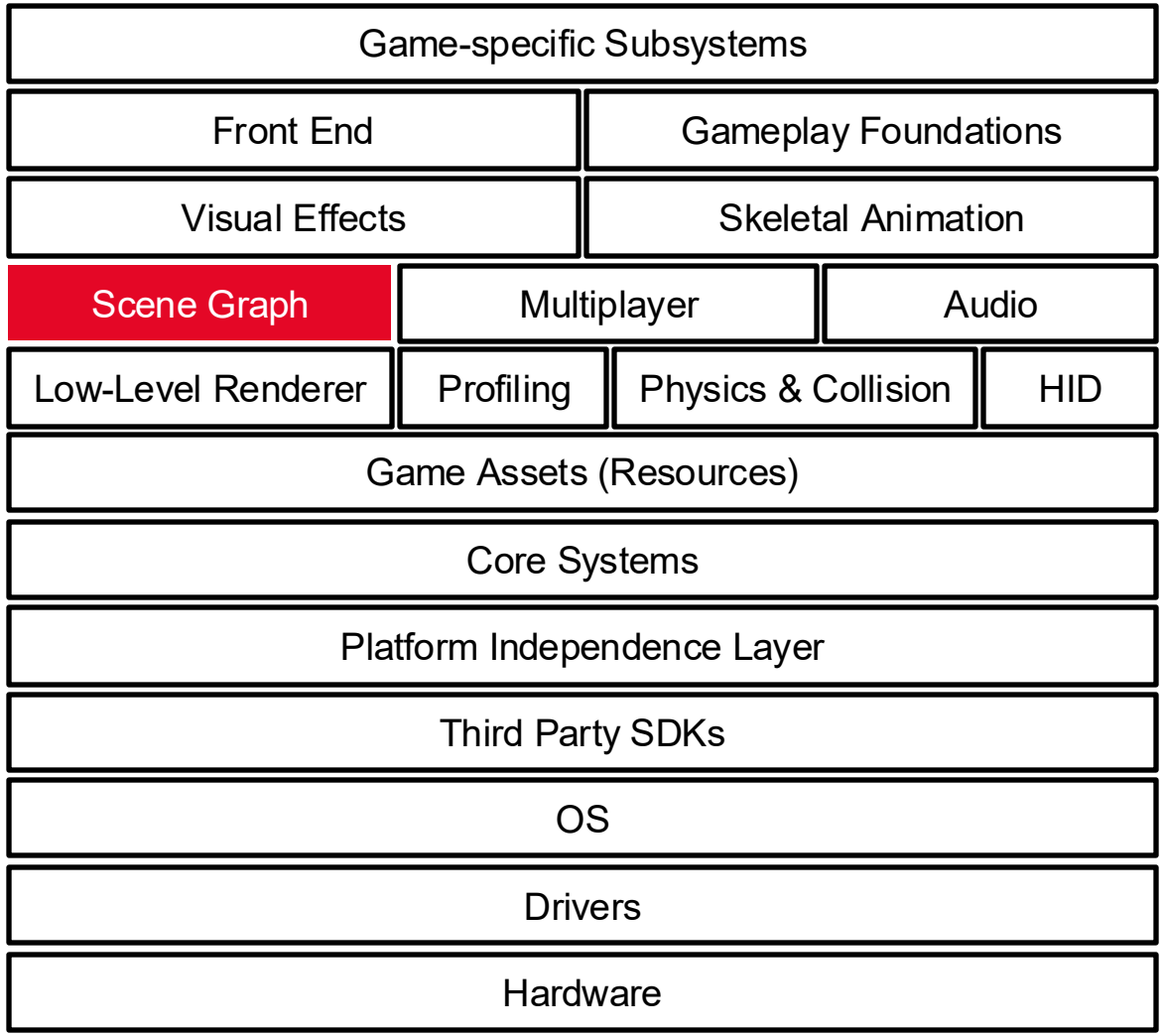
Basics

Runtime Engine Architecture

→ **Scene Graph**

- Limits the number of primitives submitted for rendering
- Based on **visibility determination**
  - Frustum culling (avoid drawing objects outside the camera's view)



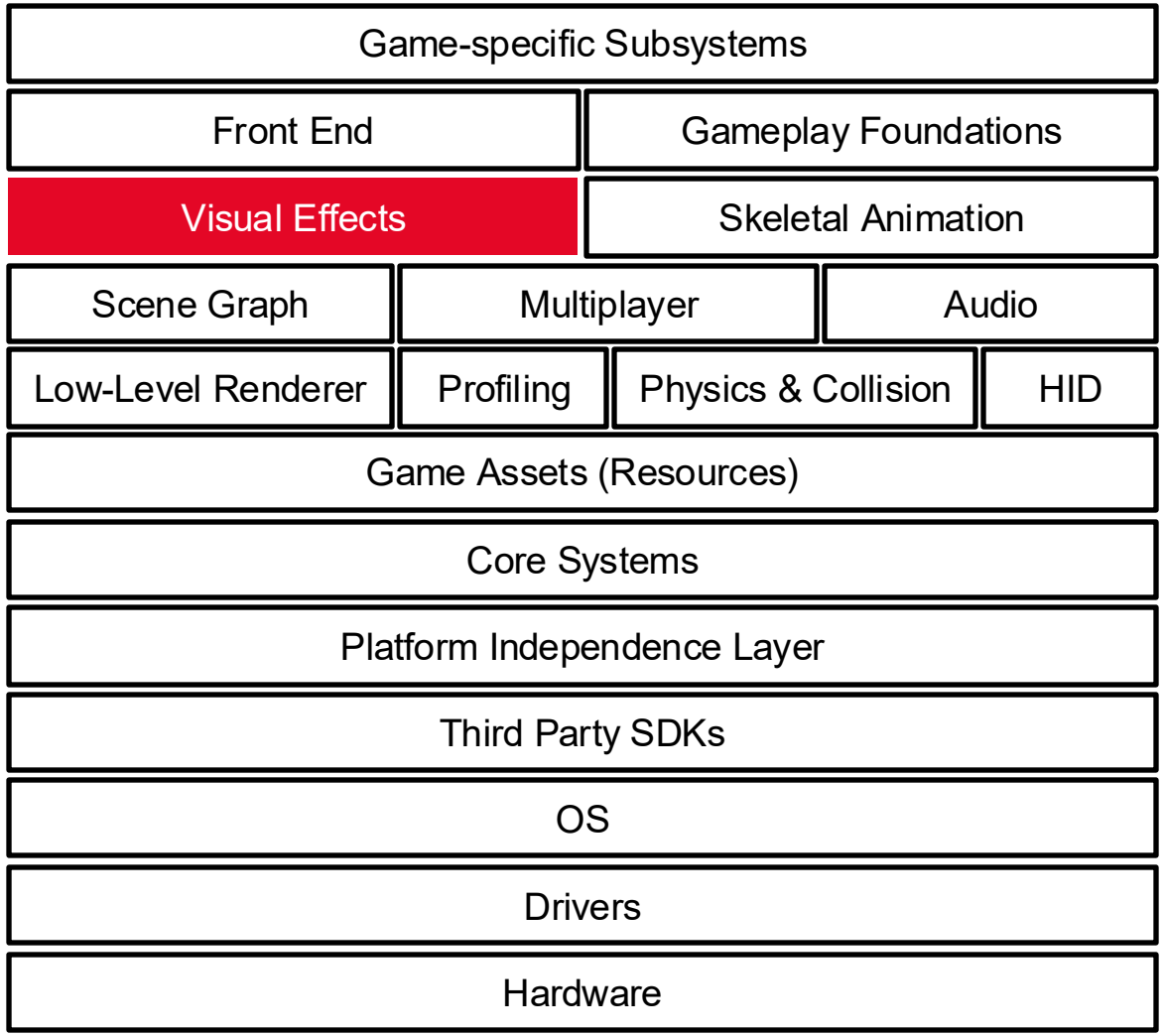


Basics

Runtime Engine Architecture

→ Visual Effects

- Particle systems (e.g., smoke, fire, water splashes)
- Decal systems (e.g., bullet holes, footprints)
- Dynamic shadows
- Post effects (applied after the scene has been rendered)

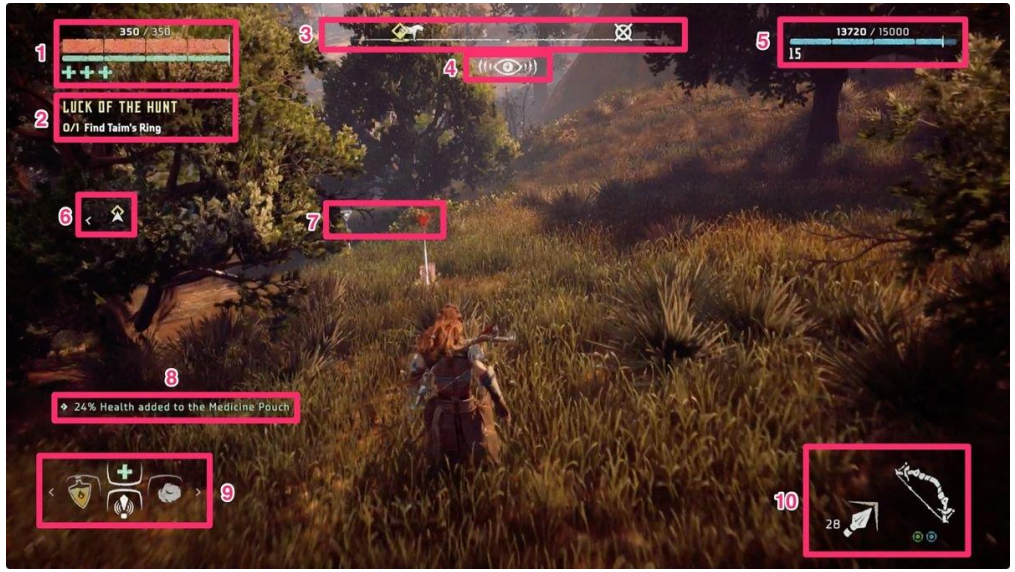


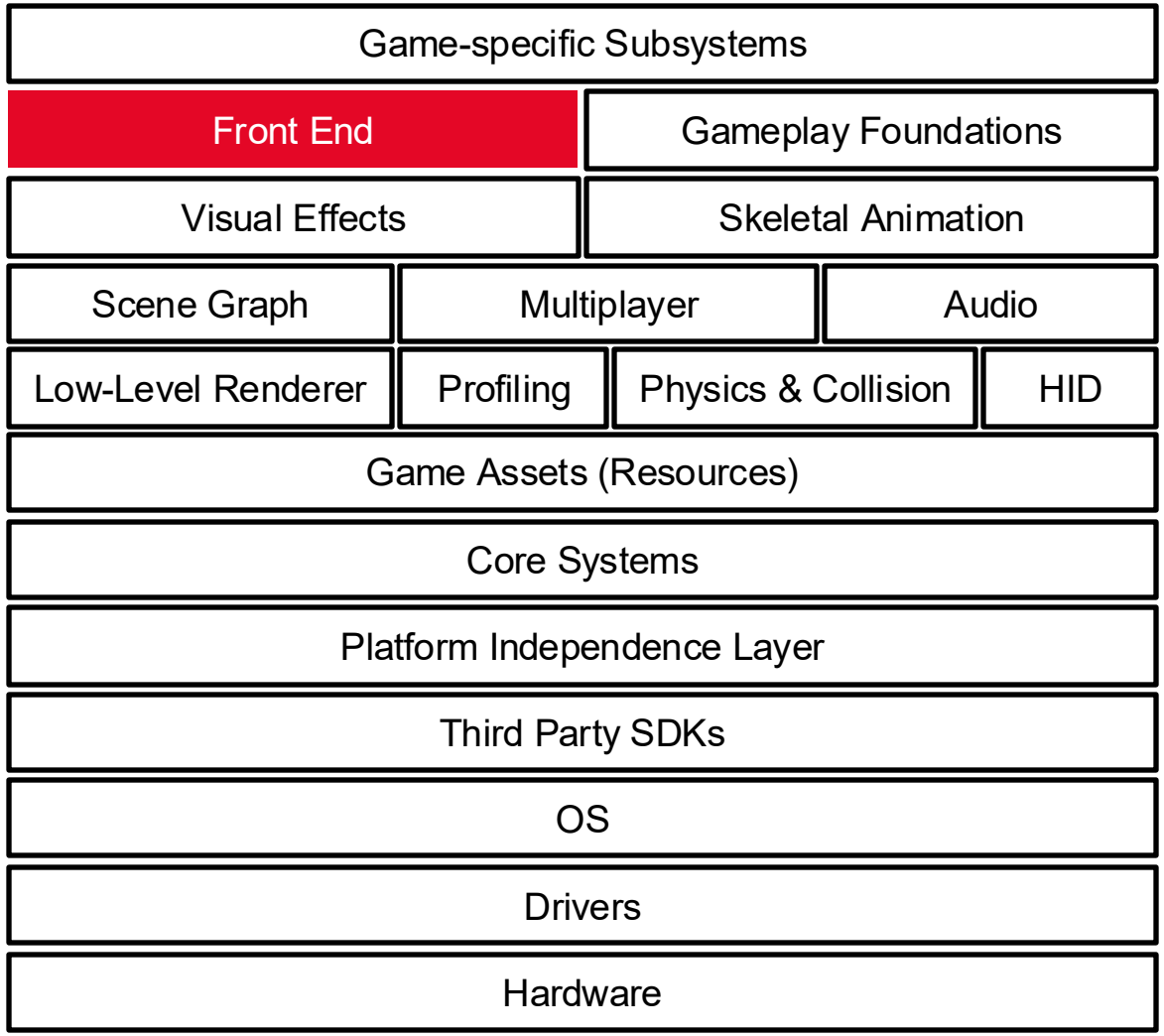
Basics

Runtime Engine Architecture

→ **Front End**

- **2D overlay**
- Heads-up display (HUD)
- In-game menus
- Graphical user interface (GUI) for inventory



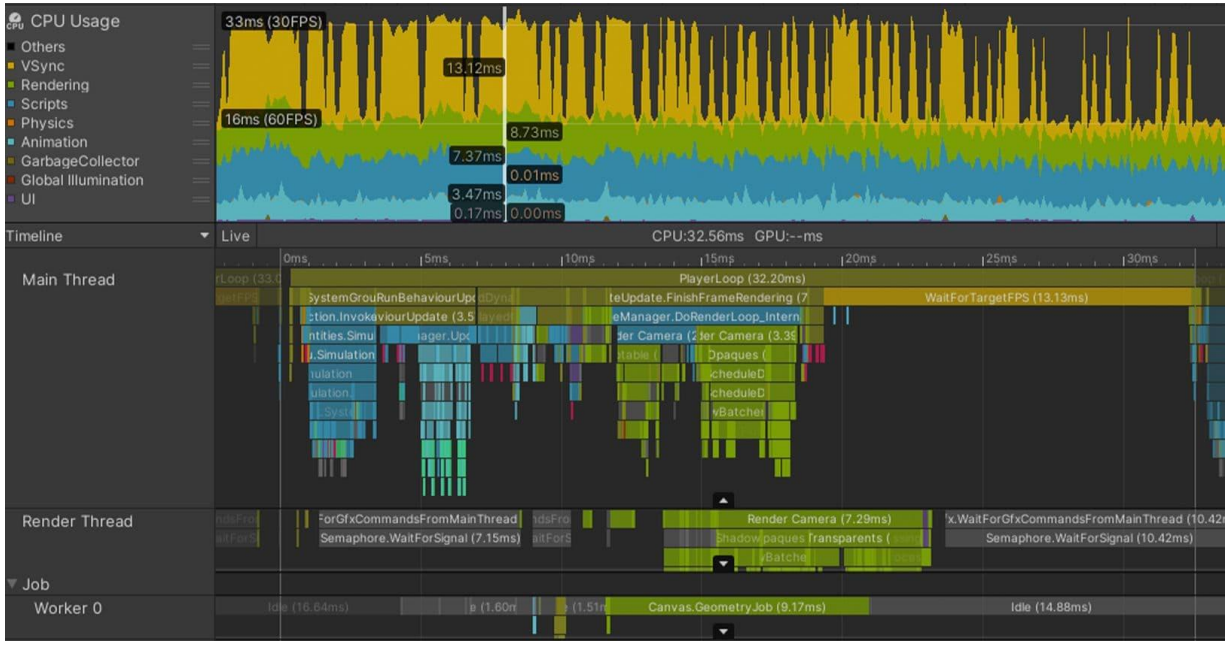


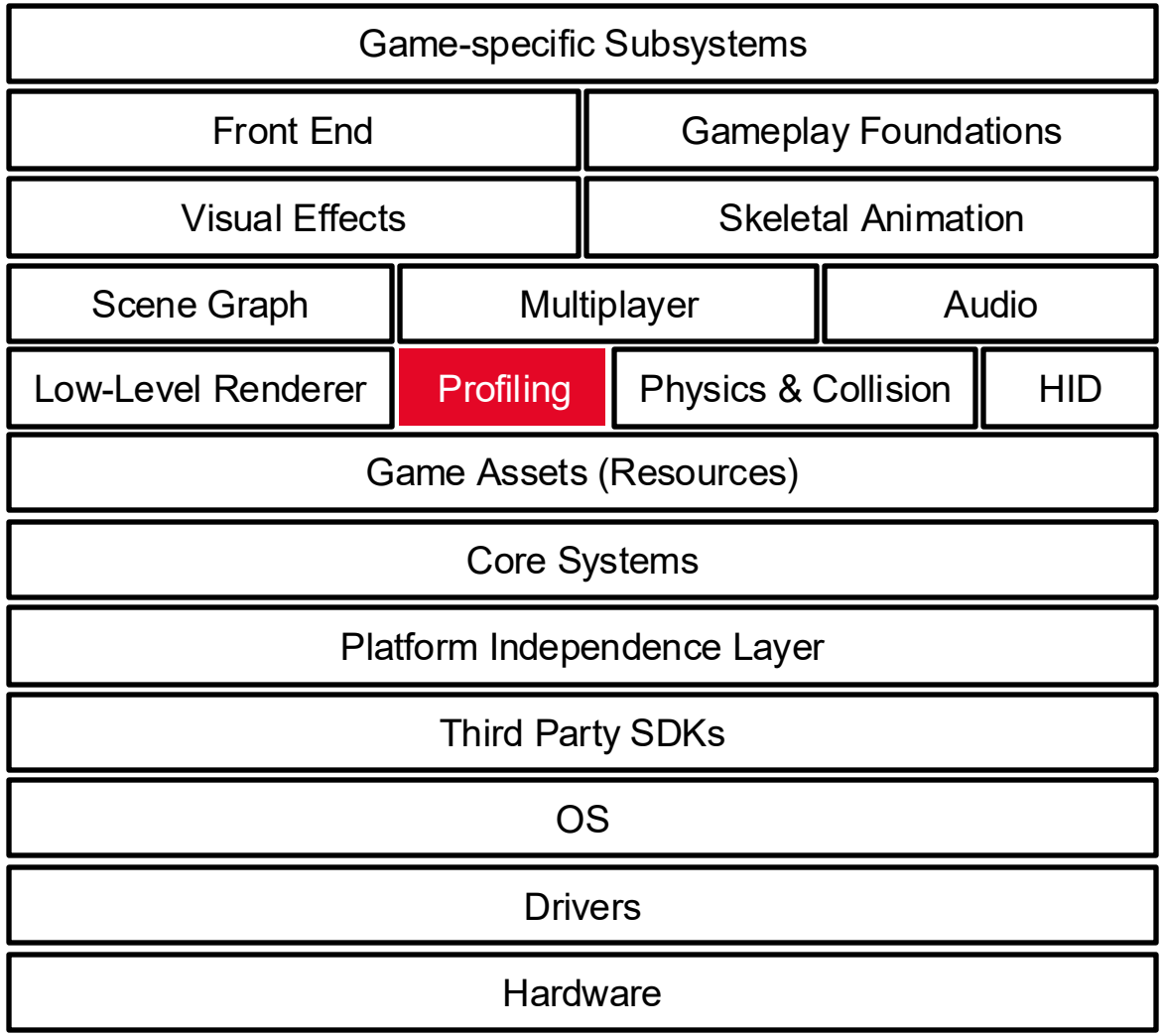
Basics

Runtime Engine Architecture

→ **Profiling and Debugging Tools**

- Performance overview
- Memory overview



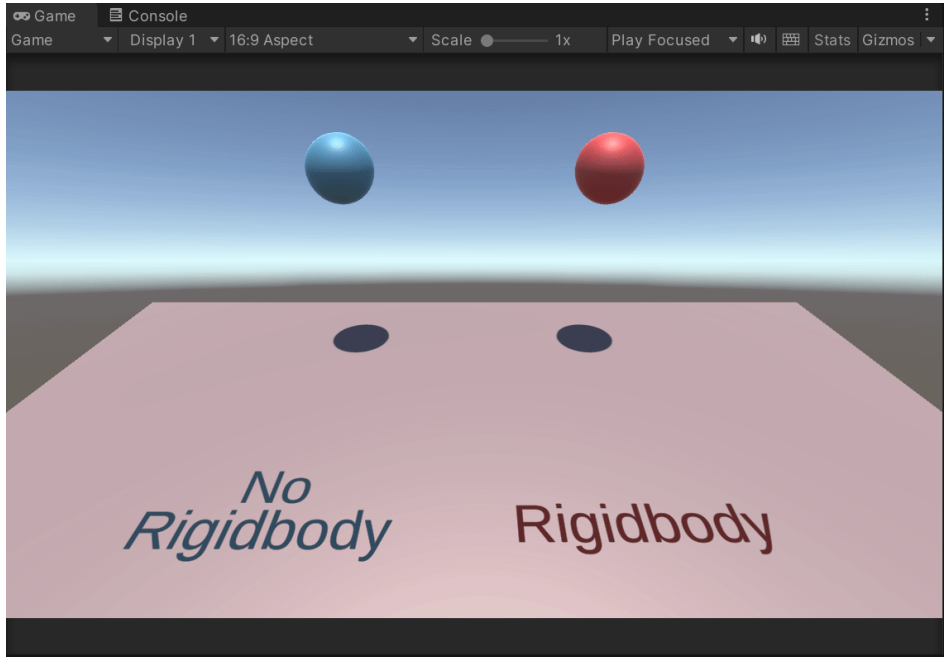


Basics

Runtime Engine Architecture

→ **Physics and Collision**

- **Motion** (kinematics) of rigid bodies and the **forces and torques** (dynamics) that cause motion to occur



Game-specific Subsystems		
Front End	Gameplay Foundations	
Visual Effects	Skeletal Animation	
Scene Graph	Multiplayer	Audio
Low-Level Renderer	Profiling	Physics & Collision	HID
Game Assets (Resources)		
Core Systems		
Platform Independence Layer		
Third Party SDKs		
OS		
Drivers		
Hardware		

Basics

Runtime Engine Architecture

→ **Human Interface Devices (HID)**

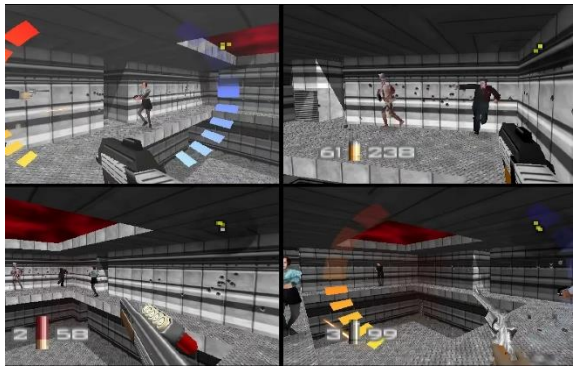
- **Player I/O (Input/Output)**
  - Keyboard
  - Mouse
  - Joypad
  - Steering wheels
  - Fishing rods
  - Dance pads
  - Wiimote

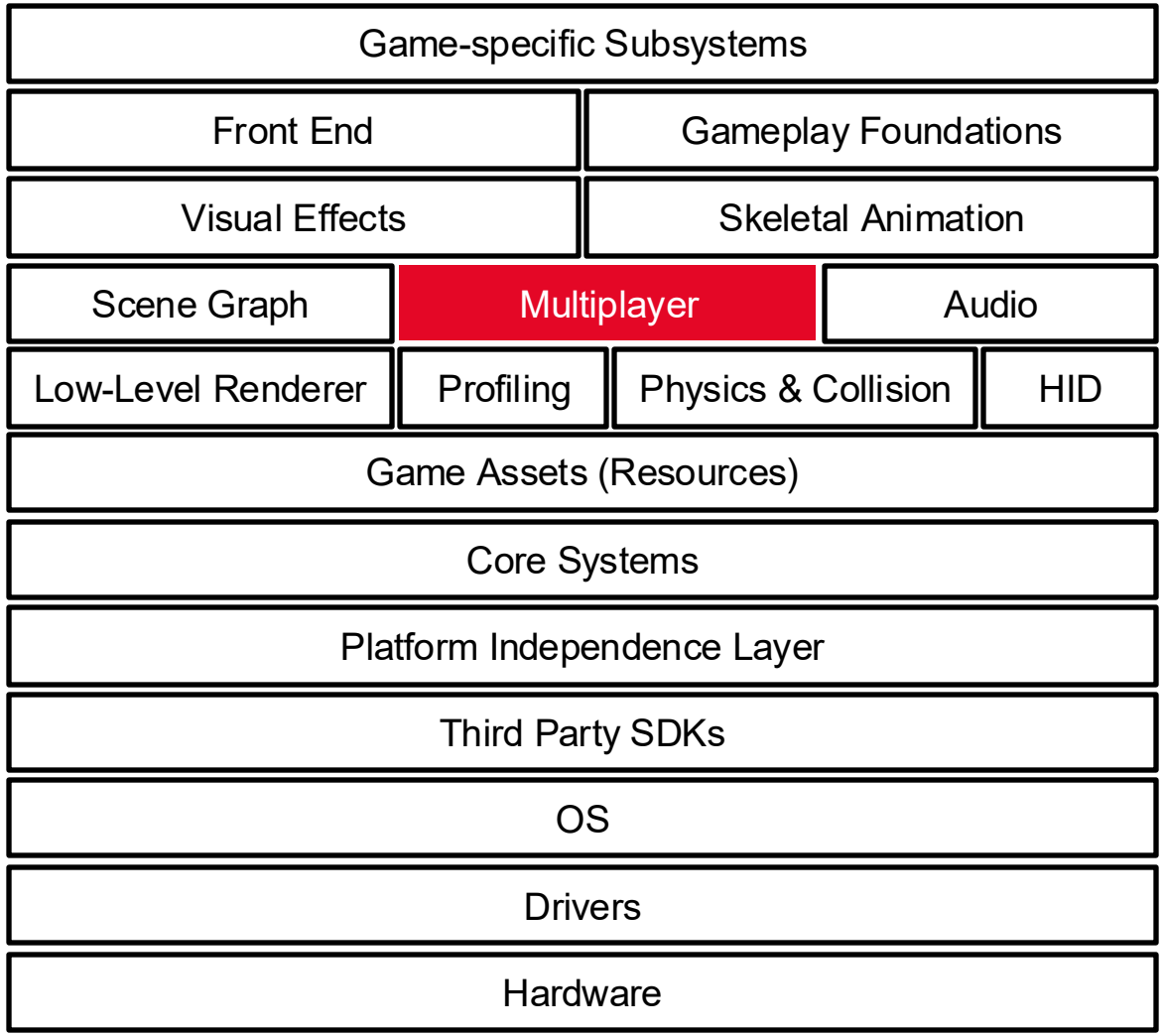
Basics

Runtime Engine Architecture

→ **Online Multiplayer/Networking**

- **Single-screen multiplayer** (e.g., Smash Bros)
- **Split-screen multiplayer** (e.g., GoldenEye 007)
- **Networked multiplayer** (e.g., Mario Kart Online)
- **Massively multiplayer online games (MMOG)**
  - 1000s of users playing simultaneously within a giant, persistent, online virtual world





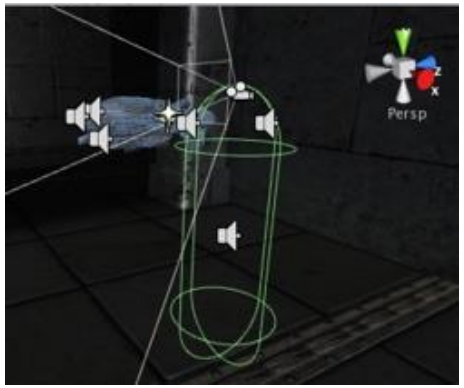
Basics

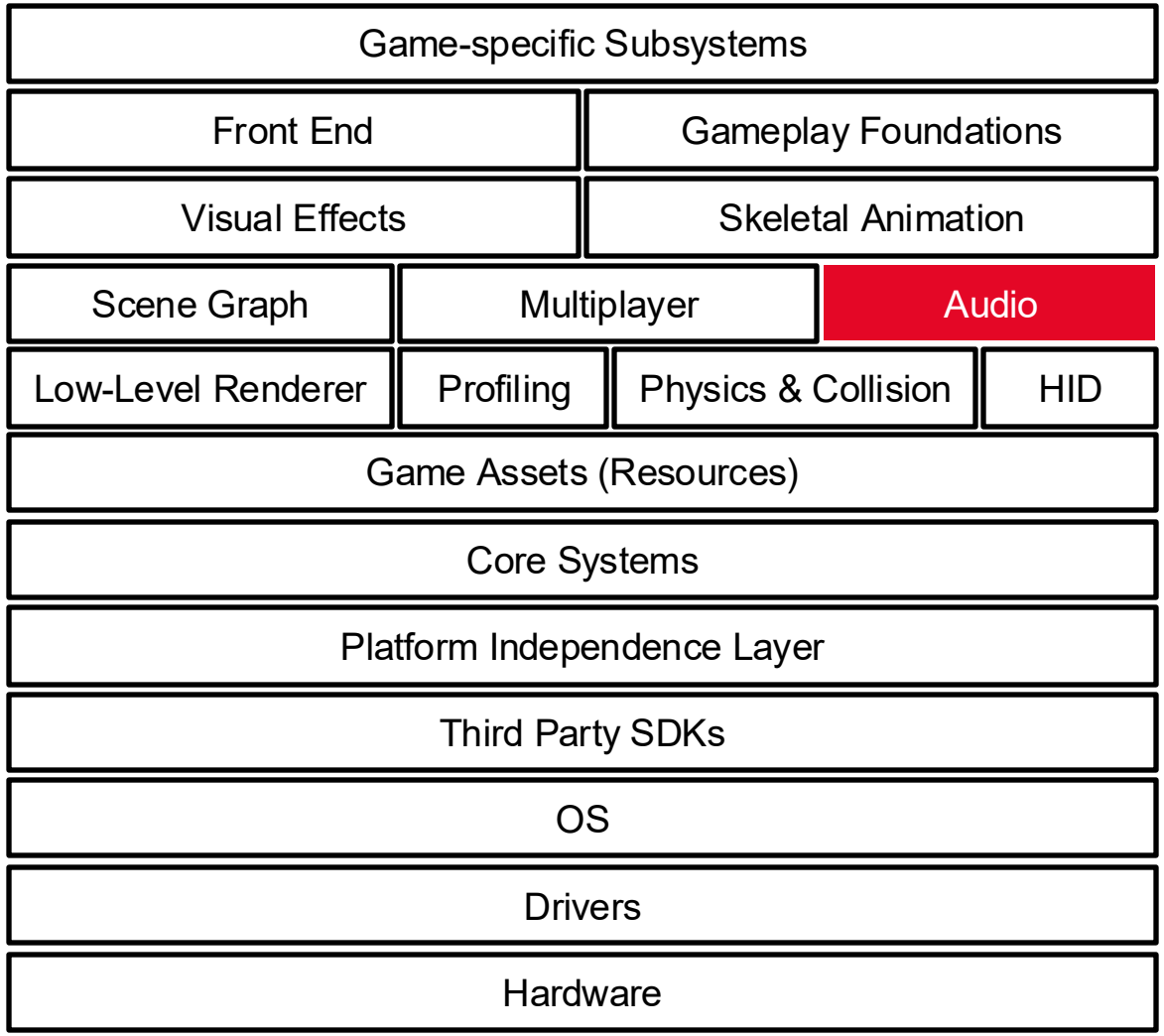
Runtime Engine Architecture

→ Audio

- Often gets less attention that rendering, physics, animation, AI, and gameplay
- No great game is complete without audio effects

- **Categories**
  - 2D audio: plays equally from both speakers (no spatial orientation); used for UI elements or music
  - 3D audio: positioned in the game world and changes based on the user's position and orientation
  - One-shot: plays only once (e.g., explosion)
  - Looping: plays continuously (e.g., fire crackling)





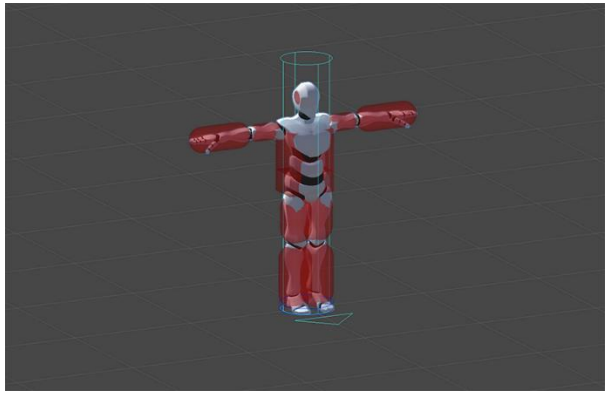
Basics

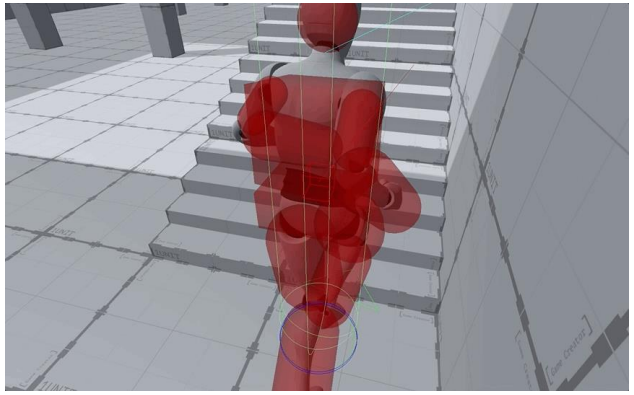
Runtime Engine Architecture

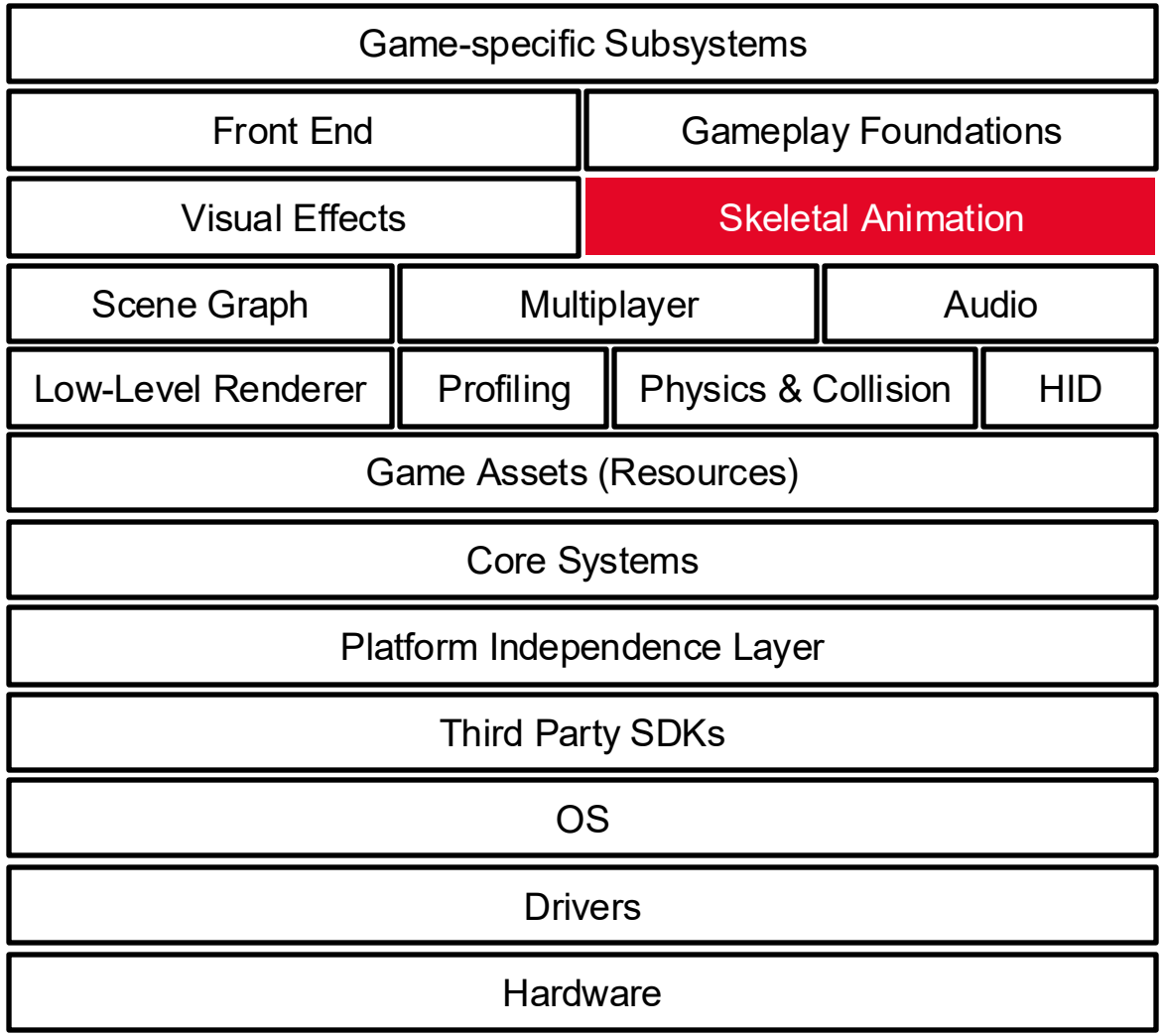
→ **Animation**

- **Basic Types**
  - Sprite/Texture animation
  - Rigid body hierarchy animation
  - Skeletal animation
  - Morph targets

- Tight **coupling between animation and physics**
  - Rag dolls







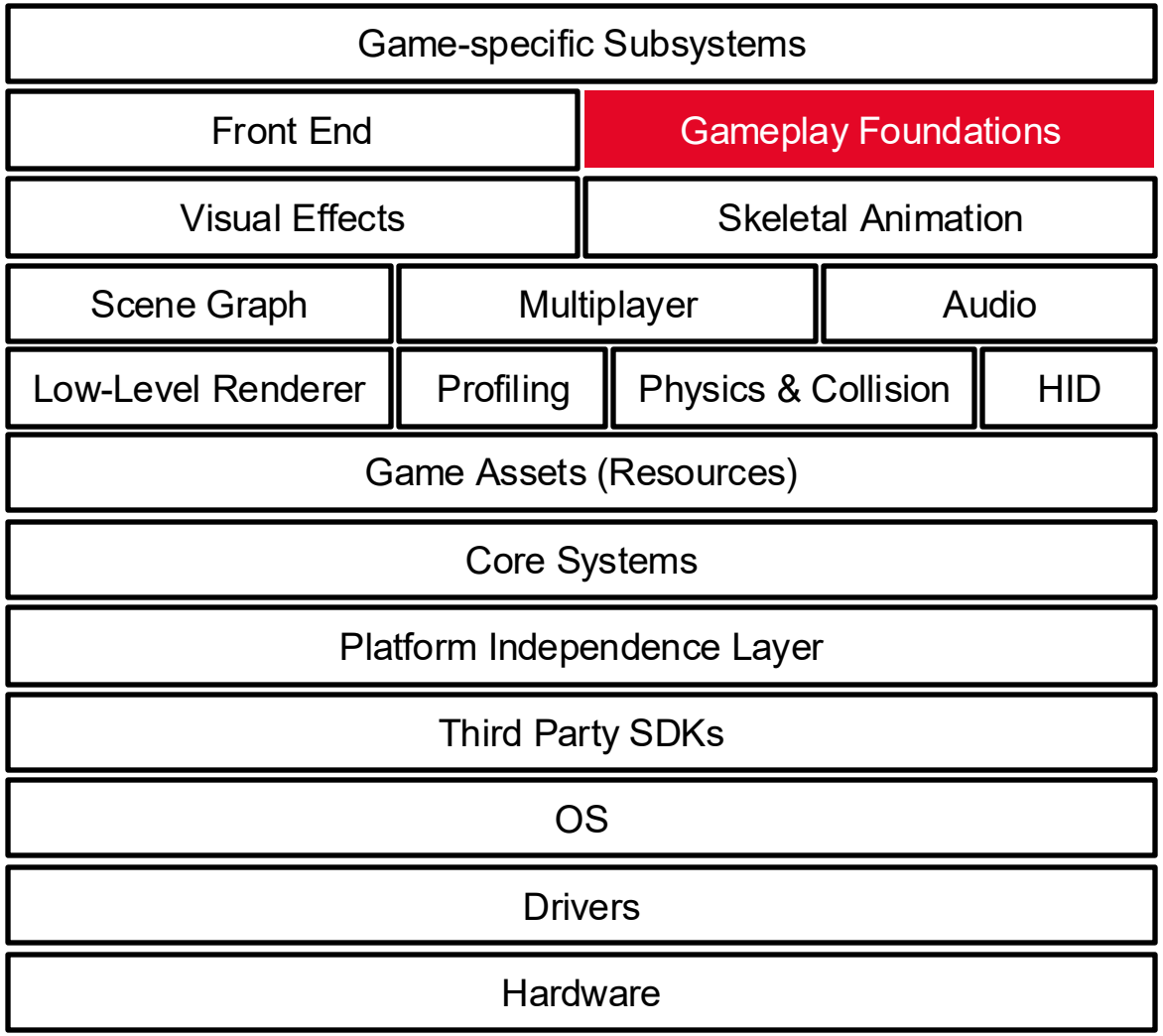
Basics

Runtime Engine Architecture

→ Gameplay Foundation Systems: Overview

- **Gameplay**
  - Action that takes place in the game
  - Rules that govern the virtual world
  - Abilities of the player (player mechanics)
- Implemented in native language or high-level scripting language
  - Unity: C#
  - Unreal: C++

```
1   using System.Collections;
2   using System.Collections.Generic;
3   using UnityEngine;
4
5   public class Rotate : MonoBehaviour
6   {
7       // Start is called before the first frame update
8       void Start()
9       {
10
11      }
12
13      // Update is called once per frame
14      void Update()
15      {
16
17      }
18  }
```

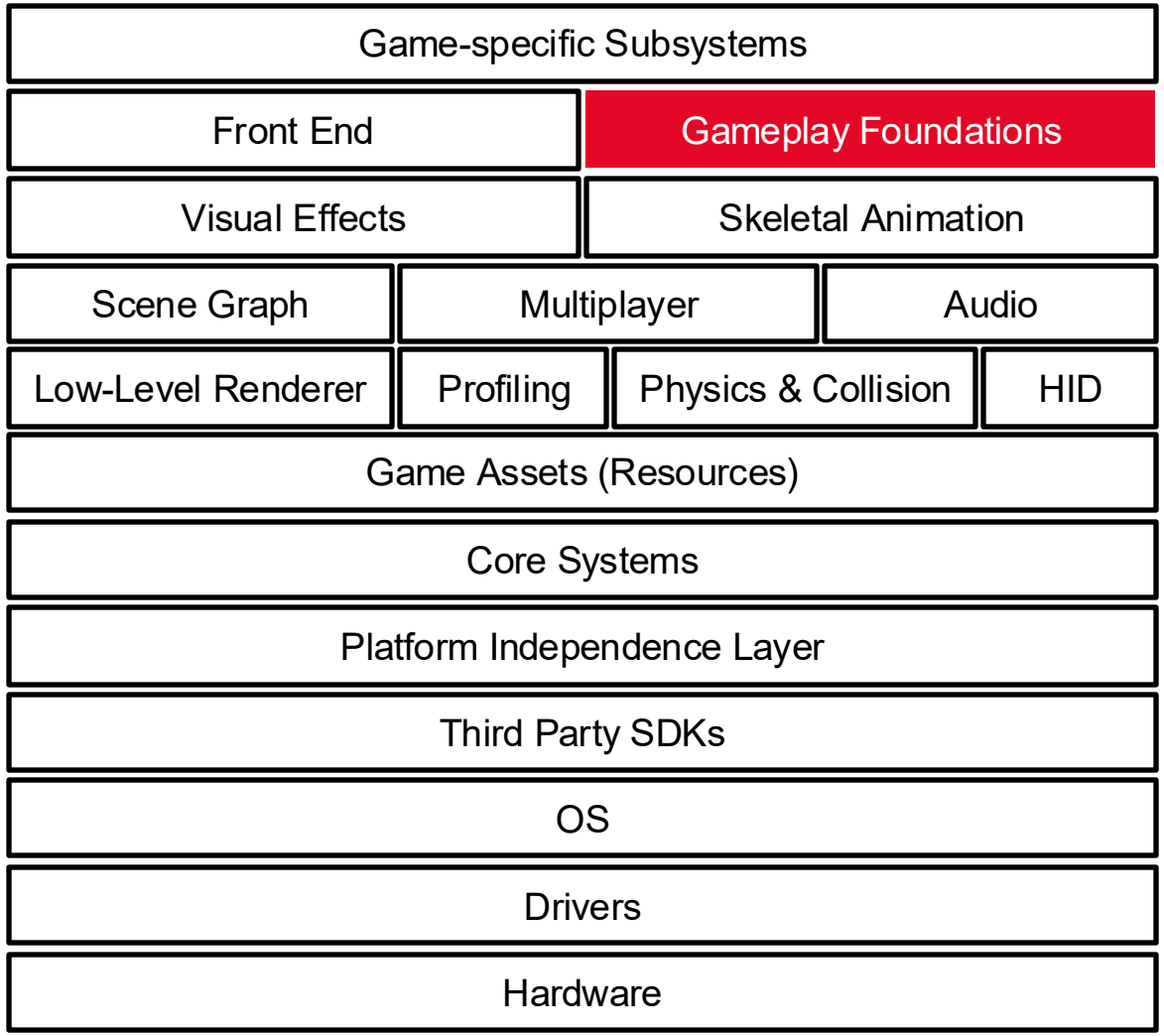


Basics

Runtime Engine Architecture

→ **Gameplay Foundation Systems: Game World and Objects**

- **Typical game objects**
  - Static background geometry (e.g., buildings)
  - Dynamic rigid bodies (e.g., chairs)
  - Player characters
  - Non-player characters (NPCs)
  - Weapons
  - Projectiles
  - Lights
  - Cameras

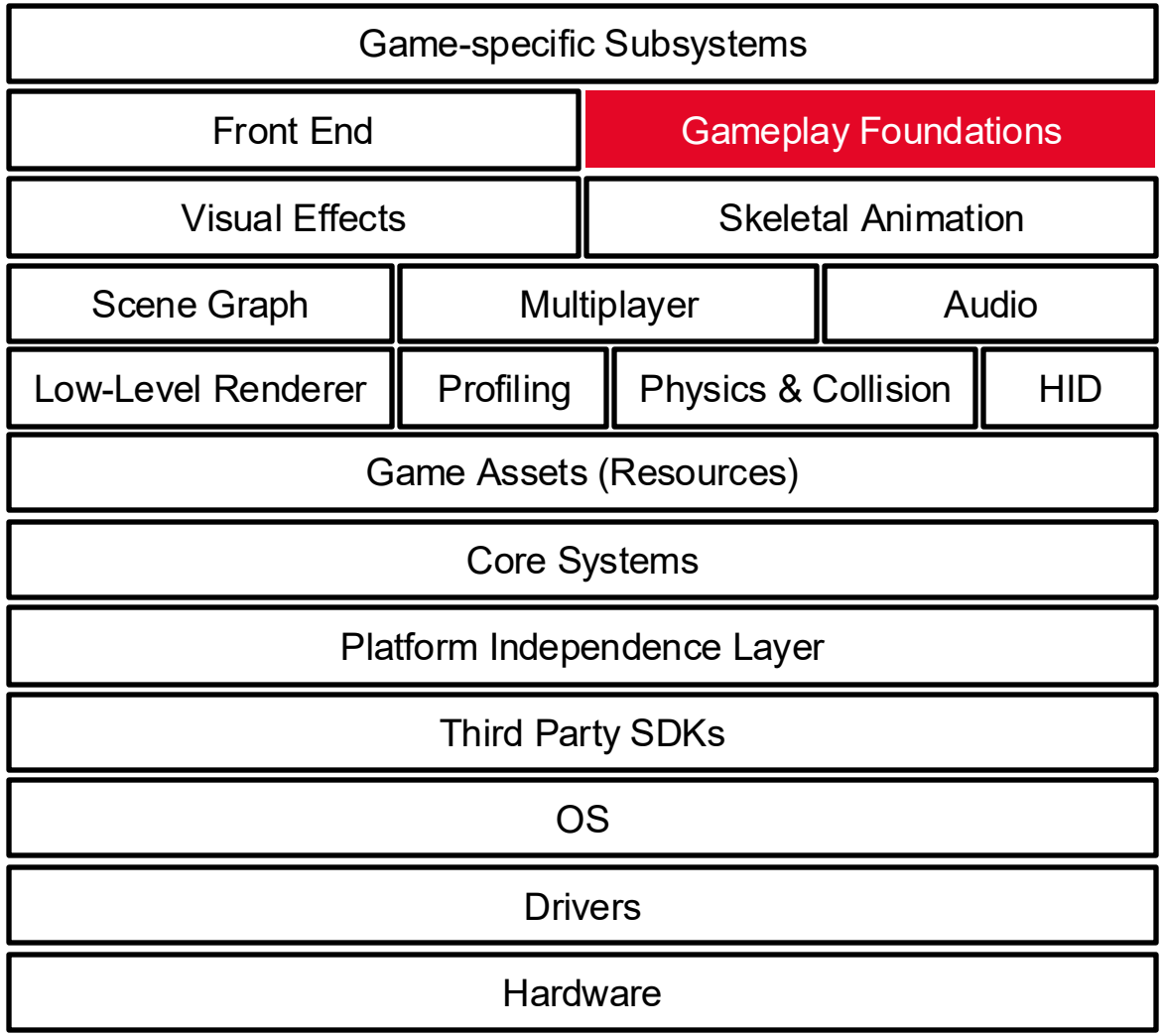


Basics

Runtime Engine Architecture

→ **Gameplay Foundation Systems: Event System**

- **Communication** between game objects
- **Sender** creates an event/message (i.e., a little data structure)
- Event is passed to the **Receiver**
- **Queues** are used for handling events in the future



Basics

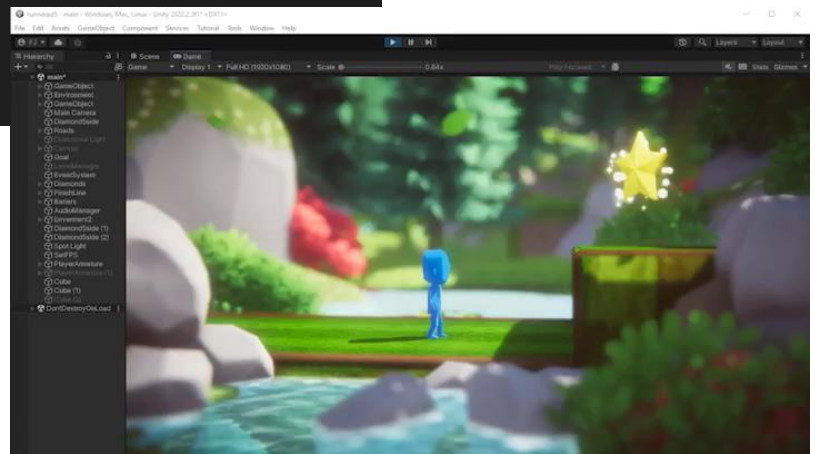
Runtime Engine Architecture

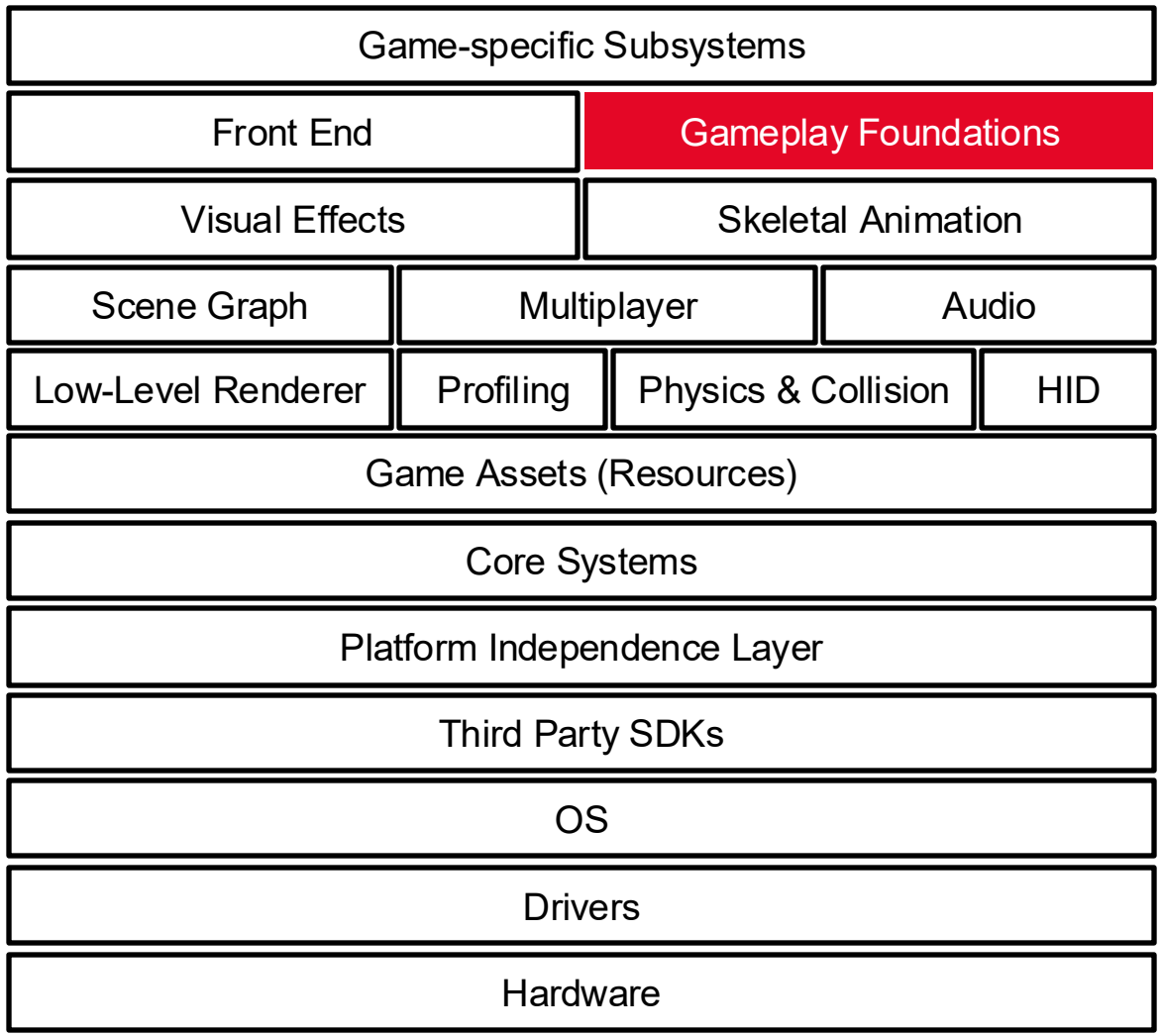
→ **Gameplay Foundation Systems: Scripting System**

- **Rapid development** of game-specific gameplay and content
- **Hot-reloading**
  - Edit code and get immediate updates in the game

```
void Update()
{

}
```



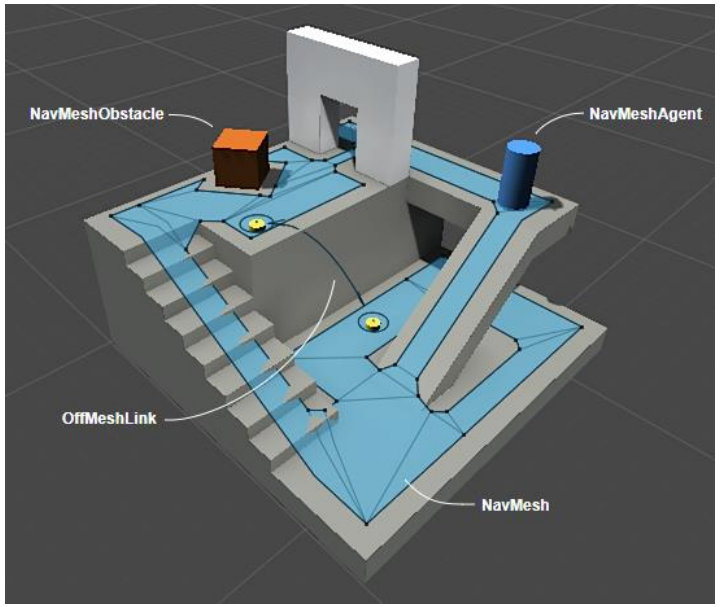


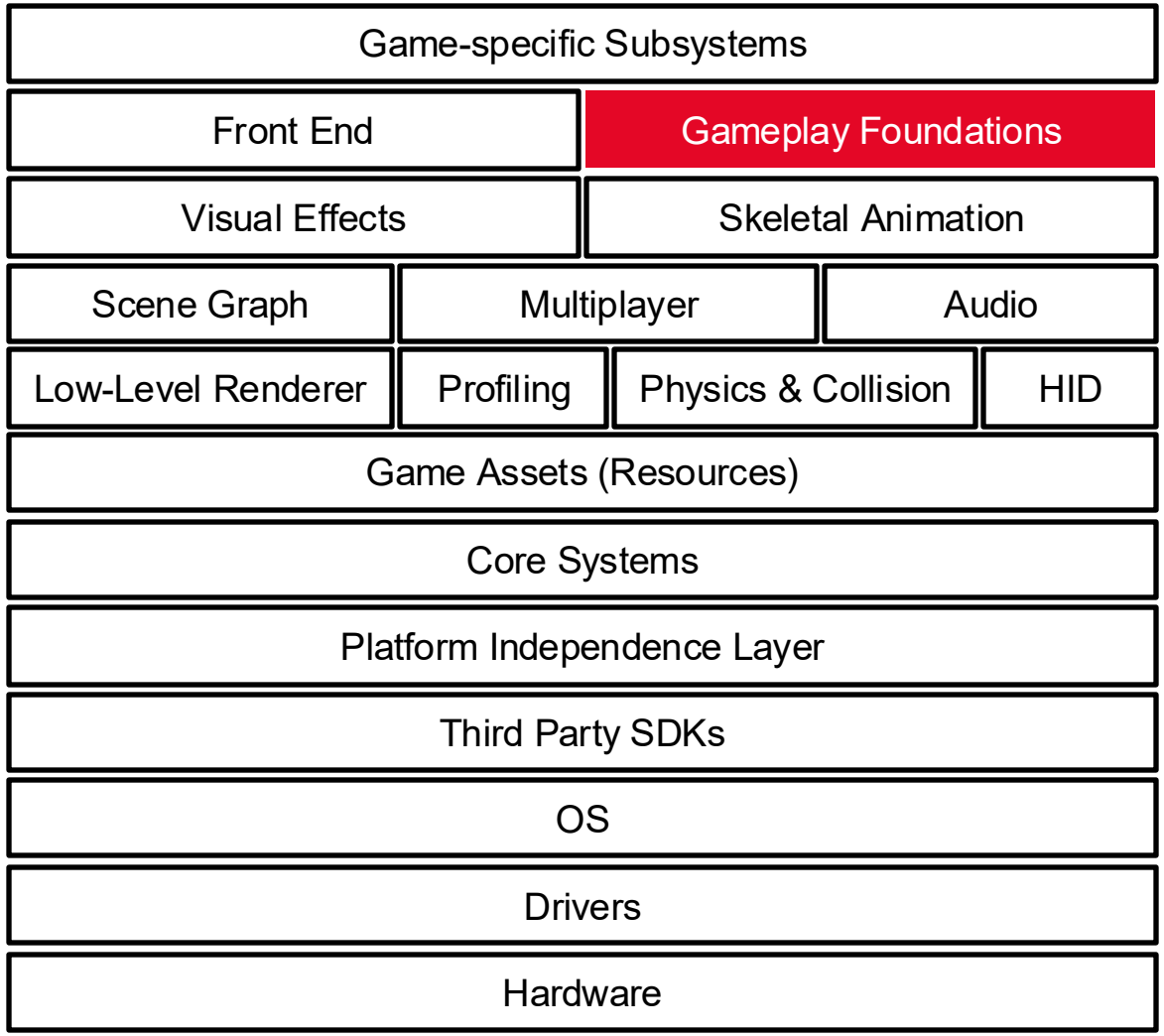
Basics

Runtime Engine Architecture

→ **Gameplay Foundation Systems: Artificial Intelligence (AI)**

- **Low-level AI building blocks**
  - Nav mesh generation
  - Path finding
  - Static and dynamic object avoidance
  - Identification of vulnerabilities
- **Reinforcement learning**



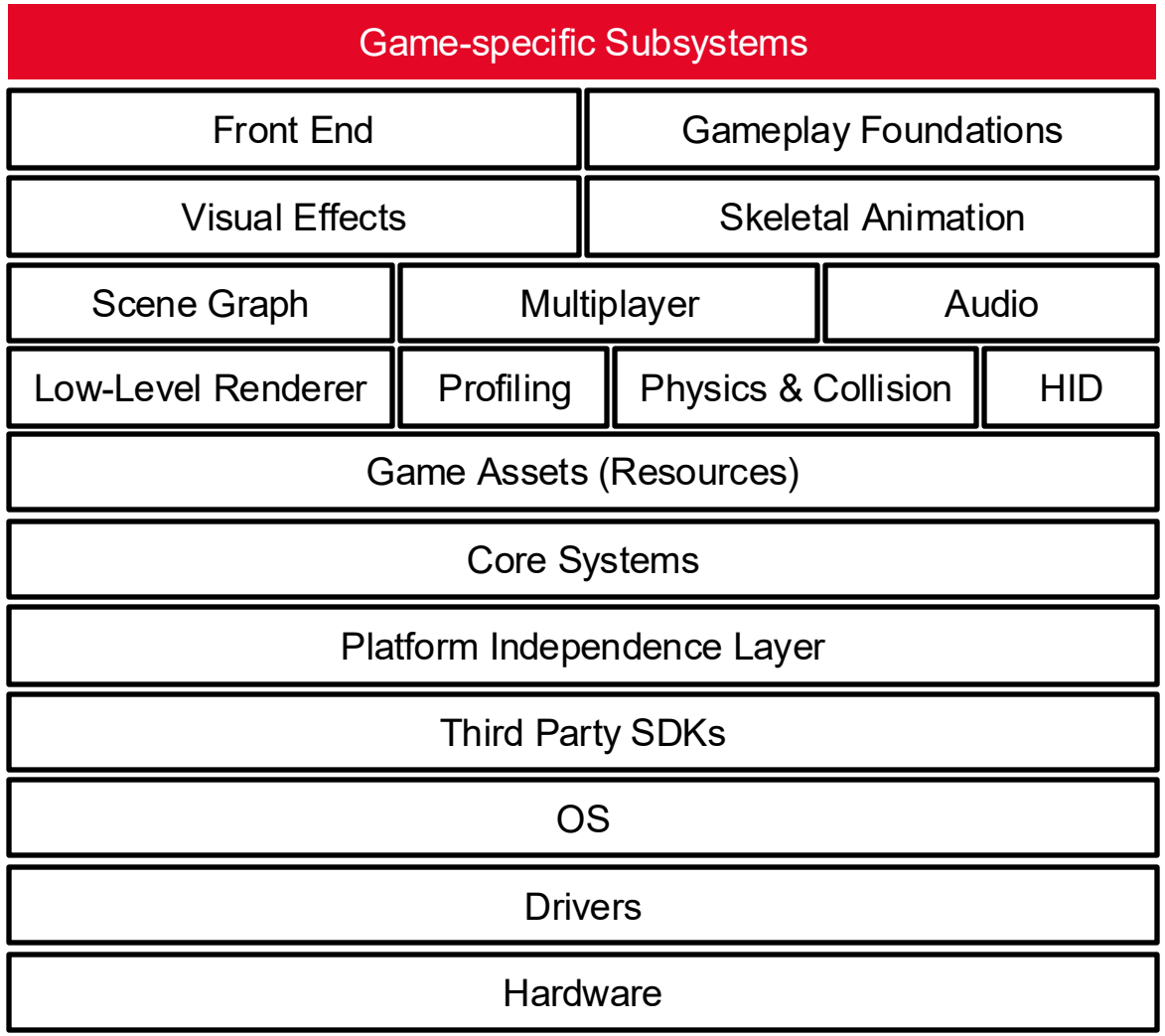


Basics

Runtime Engine Architecture

→ **Game-Specific Subsystems**

- “Line” between the game and the game engine
- Features of the game itself
  - Mechanics of the player character
  - In-game camera systems
  - AI for non-player characters (NPCs)
  - Weapon systems
  - Vehicles
  - …

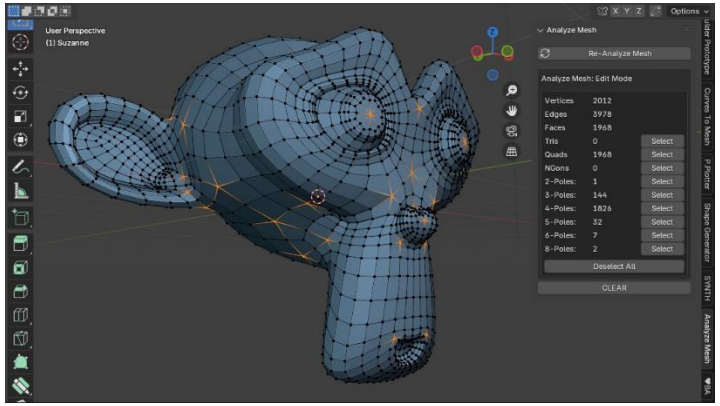


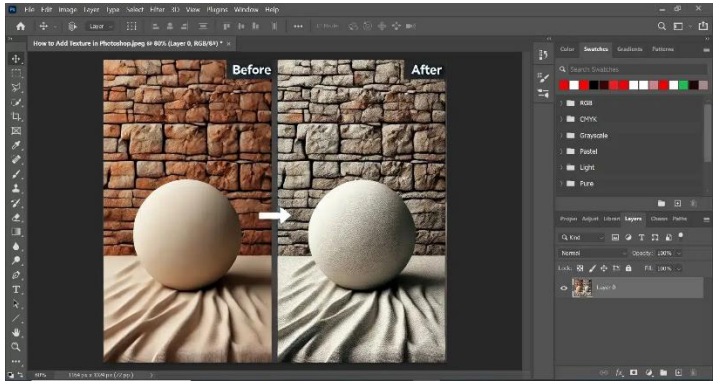
Basics

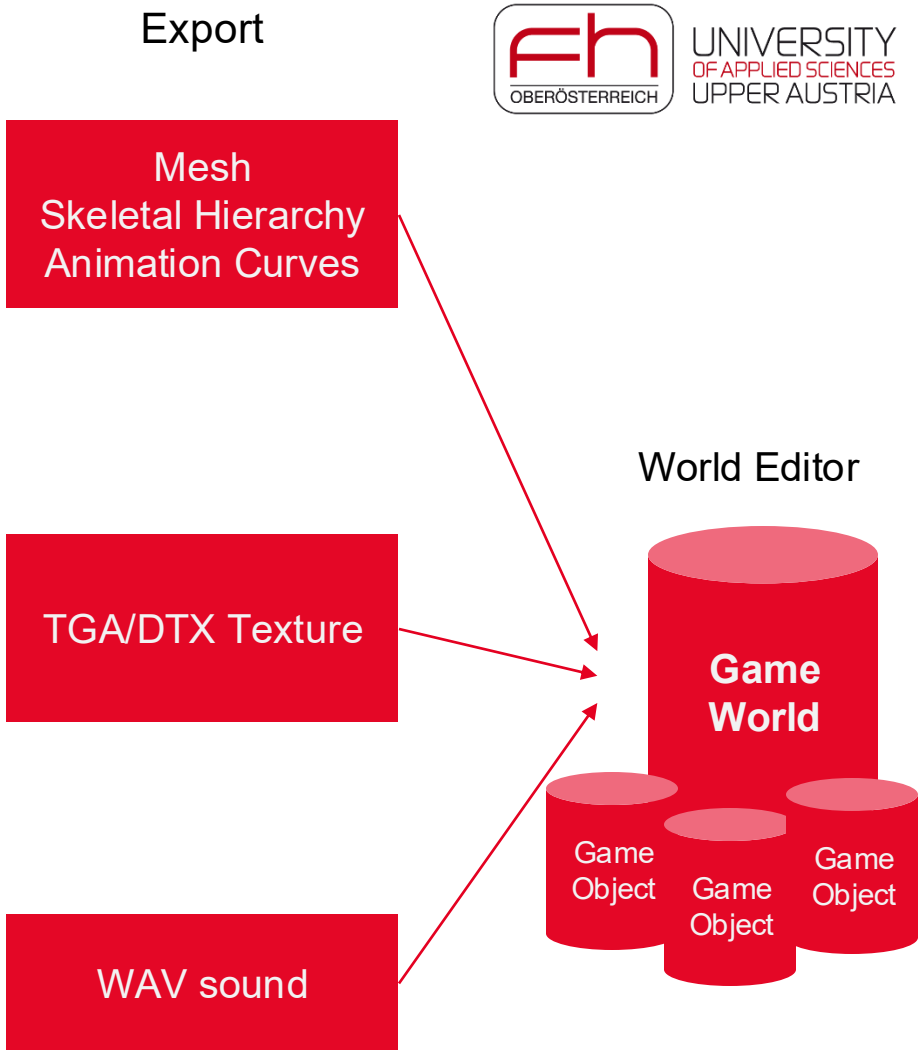
Tools and Asset Pipeline

→ **Digital Content Creation Tools**

- 3D meshes and animations
  - Autodesk Maya
  - Autodesk 3ds Max
  - Blender
  - Pixologic Zbrush
- 2D bitmaps and textures
  - Adobe Photoshop
- Particles and special effects
  - Houdini
- Audio creating and editing
  - SoundForge



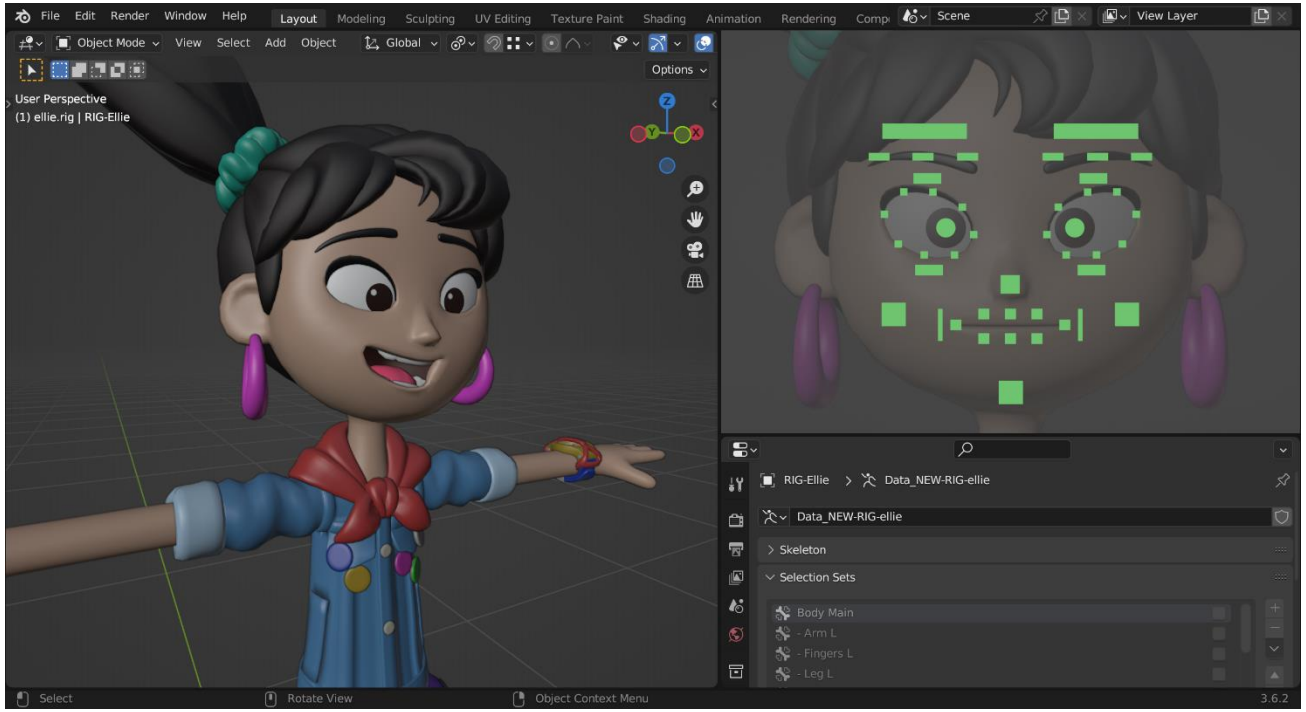




Basics

Tools and Asset Pipeline

- Digital Content Creation Tools: Export to FBX/OBJ Format
  - 3D Model/Mesh Data
  - Skeletal Animation



Basics

Tools and Asset Pipeline

- Digital Content Creation Tools: Export to WAV/MP3
  - Audio Data: mono, stereo, 5.1, 7.1, or other multi-channel configurations

Basics

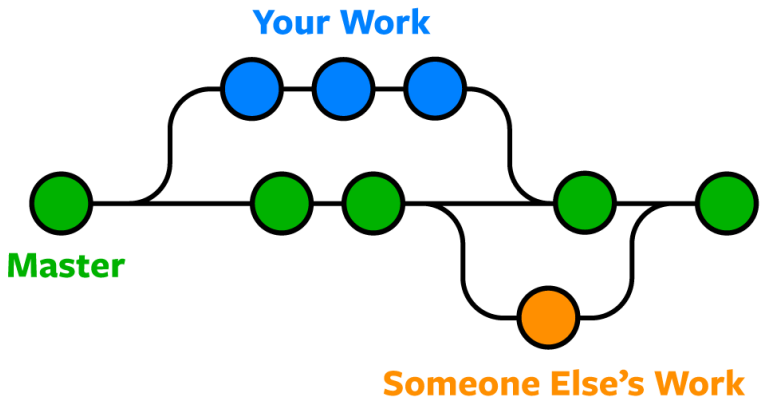
Tools and Asset Pipeline

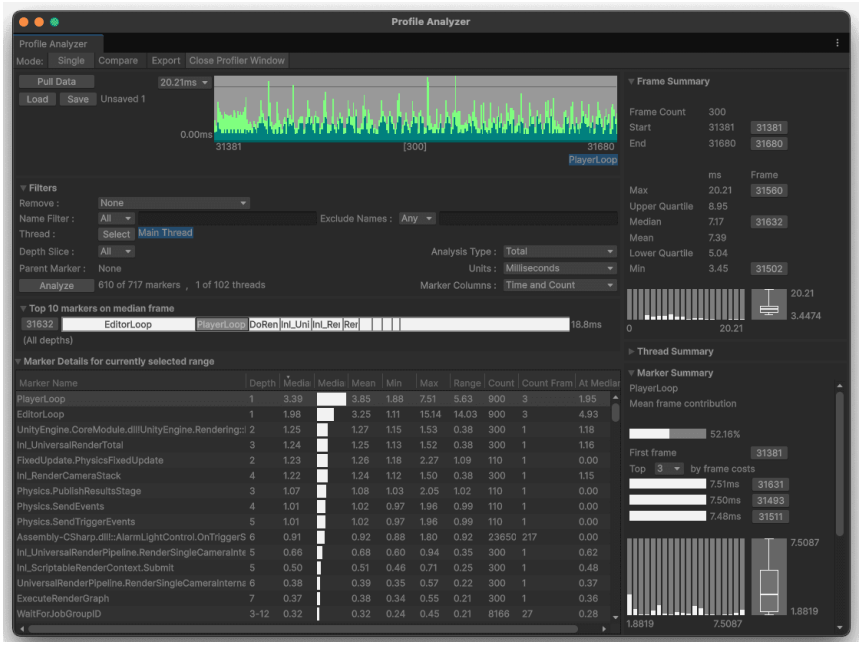
- **World Editor:** where assets come together
  - **Huge number of assets:** meshes, materials, textures, animation data, sound files
  - Assets carry **metadata**, for example:
    - A unique id that identifies the animation clip at runtime
    - The name and directory path of the source file
    - Frame range: on which frame the animation begins and ends
    - Should the animation loop or not?
    - Author
    - Creation date
    - …

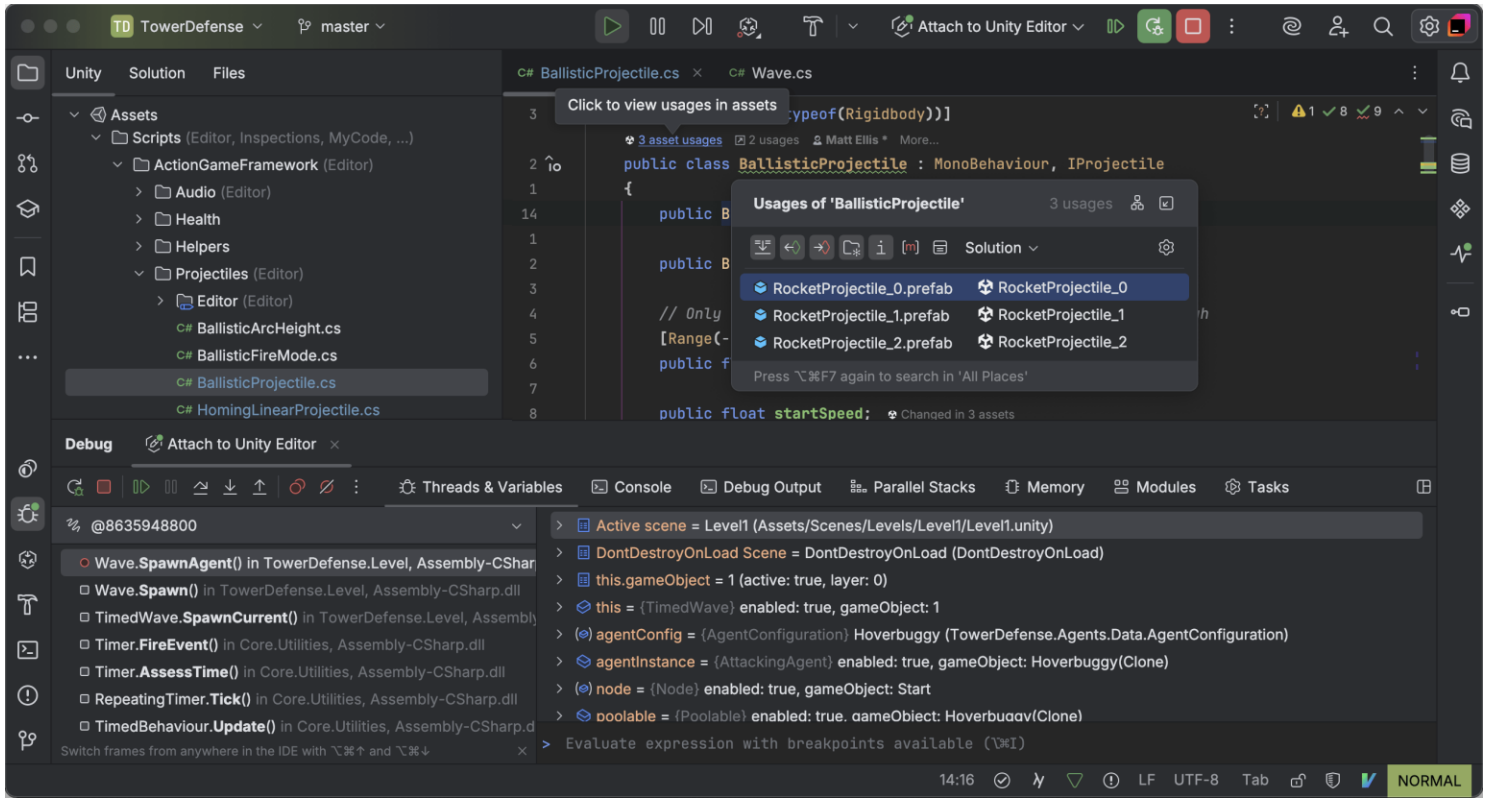
Basics

A Game Developer's Tools

- Version Control System (VCS): Git
- Integrated Development Environment (IDE)
- Profiler: performance optimization
- Crash Reporting
- …





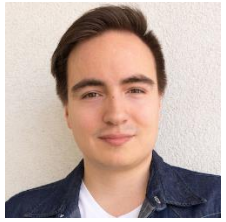


Game Engine Concepts

FH Oberösterreich
University of Applied Sciences Upper Austria

Digital Media Department

Media Technology & Design
Digital Arts
Interactive Media



Dr. Andreas Riegler
Professor of Interactive Systems
Digital Media Department
E Andreas.Riegler@fh-hagenberg.at